Multi-Agent Deep Reinforcement Learning for Cooperative Computing Offloading and Route Optimization in Multi Cloud-Edge Networks

Akito Suzuki, Masahiro Kobayashi, and Eiji Oki, *Fellow, IEEE*

***Abstract*—Edge computing is a new paradigm to provide computing capability at the edge servers close to end devices. A significant research challenge in edge computing is finding efficient task offloading to edge and cloud servers considering various task characteristics and limited network and server resources. Several reinforcement learning (RL)-based task-offloading methods have been developed, because RL can immediately output efficient offloading by pre-learning. However, these methods do not take into account clouds or focus only on a single cloud. They also do not take into account the bandwidth and topology of the backbone network. Such shortcomings strongly limit the range of applicable networks and degrade task-offloading performance. Therefore, we formulate a task-offloading problem for multi-cloud and multi-edge networks considering network topology and bandwidth constraints. We also propose a task-offloading method that is based on cooperative multi-agent deep RL (Coop-MADRL). This method introduces a cooperative multi-agent technique through centralized training and decentralized execution, improving task-offloading efficiency. Simulations revealed that the proposed method can minimize network utilization and task latency while minimizing constraint violations in less than one millisecond in various network topologies. It also shows that cooperative learning improves the efficiency of task offloading. We demonstrated that the proposed method has generalization performance for various task types by pre-training with many resource-consuming tasks.**

***Index Terms*—Edge computing, cloud computing, deep reinforcement learning, task offloading, route optimization.**

I. Introduction

With the development of communication technologies, diverse applications have emerged in various domains, e.g., healthcare, smart cities, and manufacturing. These applications are generally offloaded and processed in cloud servers because of the limitation of the computation resources of end devices (EDs), e.g., personal computers, smartphones, Internet of Things devices, and cars. This process is called cloud computing (CC). Such offloaded applications' tasks consist of the demand for computing and communication with various characteristics, e.g., traffic heavy, computing heavy, or latency sensitive. Since cloud servers are generally located far from EDs, offloading tasks to the cloud inevitably generates additional transmission latency. Therefore, CC degrades the performance of latency-sensitive tasks.

To address this issue, edge computing (EC) has been proposed to deploy computing resources at the edge servers close to EDs. Combining CC and EC provides multiple offloading choices, improving the efficiency of offloading. For example, offloading computing-heavy tasks to the cloud is effective because the cloud usually has sufficient computing resources. Similarly, offloading traffic-heavy and latency-sensitive tasks to the edge effectively shortens the transmission latency. Thus, CC and EC must be combined to improve the efficiency of various offloading tasks.

Several studies have addressed task-offloading problems for CC and EC [1], [2], [3], [4], [5], [6], [7], [8]. Current methods based on mathematical optimization still incur high computational cost. For example, Wang et al. [1] accelerated the computation time by more than 100 times using their parallel optimization framework. However, due to the computational cost, they could only evaluate it in a small network with one cloud and a few base stations (BSs) (i.e., a few edge servers). To solve this problem, reinforcement learning (RL) [9] has been gaining attention as a solution [4], [5], [6], [7], [8]. RL can immediately output the preferred task offloading by learning the relationship between input network patterns and output task offloading in advance.

Although RL can solve the problem of computation time, two issues remain. One is that these methods [4], [5], [6], [7] do not take into account CC or target only the network with a single cloud. As mentioned above, CC and EC need to be combined to improve task-offloading efficiency. Moreover, a typical network has multiple clouds. The other issue is that these methods [4], [5], [6], [7], [8] do not take into account the bandwidth capacity and backbone-network topology. Many studies attempted to minimize task latency by shortening the path that offloaded tasks take. When a control system does not take into account bandwidth capacity, some links may become congested due to the concentration of tasks. Therefore, the system should obtain the task offloading that minimizes latency while satisfying all link-bandwidth constraints. Some

Manuscript received 2 October 2022; revised 28 February 2023 and 11 April 2023; accepted 12 April 2023. Date of publication 18 April 2023; date of current version 12 December 2023. This journal paper is an extension of the conference version, which has been accepted by IEEE ICC 2022 [DOI: 10.1109/ICC45855.2022.9838540]. The associate editor coordinating the review of this article and approving it for publication was Y. Diao. *(Corresponding author: Akito Suzuki.)*

Akito Suzuki and Masahiro Kobayashi are with the NTT Network Service Systems Laboratories, NTT Corporation, Tokyo 180-8585, Japan (e-mail: akito.suzuki@ntt.com; masahiropk.kobayashi@ntt.com).

Eiji Oki is with the Graduate School of Informatics, Kyoto University, Kyoto 606-8501, Japan.

Digital Object Identifier 10.1109/TNSM.2023.3267809

© 2023 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

of the above studies assumed a typology that is not practical as a backbone network, e.g., a full-mesh network.

With RL-based task-offloading methods, multi-cloud and network topology are challenging to consider because route optimization requires many variables. Even a minimal network with five nodes requires hundreds of variables to optimize all routes between nodes. The performance of RL drastically worsens as the number of variables increases [10]. Therefore, computing offloading and route optimization are difficult to jointly solve by using RL-based methods. Since the joint problem is a non-deterministic polynomial time (NP)-hard mixed integer linear problem, it is also difficult to solve by mathematical optimization.

The key solution to this problem is that mathematical optimization can quickly solve the route-optimization problem. Therefore, we develop a method for combining RL and mathematical optimization. This method calculates the optimal computing offloading by RL and optimal routing by mathematical optimization. However, not all constraints may be satisfied when two methods are calculated individually. Therefore, we propose a combining method that uses the solution of mathematical optimization for RL reward, enabling all objectives and constraints of two methods to be considered. This method is inspired by the extendable integrated control architecture [11] that can coordinate multiple control algorithms prepared for each control metric.

Improving RL is another critical issue. Multi-agent RL (MARL) is effective in solving more complex problems. It is a system of multiple agents interacting within a common environment. Each agent works with the other agent(s) to achieve a team reward. The learning cost of each agent can be reduced by assigning each agent to each task. However, when training decentralized and independent agents to optimize the team reward, each agent faces a non-stationary learning problem [12]. An example of this problem is that when each agent independently acts simultaneously, all tasks will be allocated on the light-load server, resulting in server overload. To avoid a non-stationary problem, Zhan et al. [4] developed an algorithm combining MARL and game theory. However, the actions of decentralized agents based on game theory fall into a sub-optimal solution of the Nash equilibrium.

We formulate an optimal task-offloading problem for multi-cloud and multi-edge networks considering network topology and bandwidth constraints. We handle the task offloading of edge-to-edge and edge-to-cloud. Each task arriving at the nearest edge is either processed at the nearest edge or offloaded to the neighboring edge or cloud. We define optimal offloading as a solution that maximizes server- and link-resource efficiency and minimizes task latency while satisfying the constraints of server and link capacity and task latency. The decision variables are the computing-resource allocation of tasks and routing between the ED and allocated server. We also propose a task-offloading method that is based on cooperative multi-agent deep RL (**Coop-MADRL**). We assign an agent at each edge that has learned the optimal task offloading. We introduce a cooperative multi-agent technique in which several agents jointly optimize a single reward in a centralized training and decentralized execution manner. This can prevent the non-stationary problem and improve task-offloading efficiency. The proposed method combines Coop-MADRL and mathematical optimization to take into account network topology and bandwidth constraints in the task-offloading problem. We use the solution of mathematical optimization in the learning process of Coop-MADRL. The proposed method calculates the optimal computing offloading by RL and the optimal routing by mathematical optimization. Therefore, our method can handle numerous routing variables without approximating or reducing them.

The main contributions can be summarized as follows:

- We formulate an optimal task-offloading problem for multi-cloud and multi-edge networks considering network topology and bandwidth constraints to find the optimal solution that maximizes server-resource, link-resource, and task-latency efficiency while satisfying the constraints of server capacity, link capacity, and task latency.
- We propose a task-offloading method that is based on Coop-MADRL. We assign task-offloading control of each edge to each agent. Agents jointly optimize a single reward in centralized training and decentralized execution. This can prevent the non-stationary problem and improve task-offloading efficiency. The method combines Coop-MADRL and mathematical optimization to take into account network topology and bandwidth constraints in the task-offloading problem.
- We introduce a generalized task model representing various task types, e.g., traffic-heavy, computing-heavy, and latency-sensitive tasks. This model can evaluate diverse offloading methods under mixed task conditions generated by various applications, such as healthcare, smart cities, and manufacturing.
- We evaluated the effectiveness of the proposed method through simulations in terms of performance, network-topology dependency, computation time, and scalability regarding the number of tasks. The simulation results indicate that our method can find a solution that minimizes network utilization and task latency while minimizing constraint violations in less than one millisecond in various network topologies.
- We also evaluated the generalization performance of our method for unknown task patterns. The simulation results indicate that our method has generalization performance for various task types by pre-training with many resource-consuming tasks.

This paper is an extension of the conference version [13]. The extension can be summarized as follows:

- The primary extension is a comprehensive evaluation of the proposed method. We evaluated its performance, network-topology dependency, computation time, scalability, and generalization performance for unknown task patterns.
- We prepared three new comparison methods for evaluation, including exhaustive search and heuristic-based methods. We also prepared more diverse tasks for evaluation. In the conference version, we selected the task-computation request size from several discrete values. In

TABLE I
COMPARISON OF RELATED STUDIES

Studies	Network Properties				Method Properties		Evaluation Properties	
	# of Edges	# of Clouds	Topology	Access Network	Methodology	Cooperation	# of Task Types	Comprehensiveness
[1]	Several	Single	-	Wireless	ADMM	✓	Single	Limited
[2]	Dozens	Single	-	Wireless	SA	-	Single	Limited
[3]	Several	Single	-	Wireless	Pipeline Strategy	-	Single	Limited
[4]	Several	-	-	Wireless	MADRL	-	Single	Limited
[5]	Dozens	-	-	Wireless	MADRL	✓	Single	Limited
[6]	Several	Single	-	Wireless	MADRL	✓	Single	Limited
[7]	Several	Single	-	Wireless	MADRL	✓	Single	Limited
[8]	Several	Multi	-	Wireless	MADRL	-	Single	Limited
Ours	Dozens	Multi	✓	-	MADRL	✓	Multi	✓

this study, however, we extended the computing size to continuous values.

- We improved the implementation to reduce the training time and improve performance. The main improvement is parallelization, which can speed up the training by several times.
- We simplified the efficiency function in the reward calculation of RL to improve the clarity of the learning strategy.

The rest of the paper is organized as follows. Section II describes related work. Section III describes the formulation of the task-offloading problem. Section IV briefly reviews RL. Section V describes the proposed Coop-MADRL-based task-offloading method. Section VI describes the evaluation of its performance, and Section VII concludes the paper.

II. RELATED WORK

Several studies have addressed task-offloading problems for CC and EC [1], [2], [3], [4], [5], [6], [7], [8]. In general, CC has sufficient computing resources but inevitably increases network latency. On the other hand, EC can reduce network latency at the expense of having sufficient computing resources. These studies aim to optimize the task-offloading when considering the characteristics of heterogeneous computing resources. They sometimes refer to fog computing instead of CC or EC, or assume multiple layers in the edge network. When an edge fog or a two-tier edge network considers the heterogeneous resource characteristics, we can see them as the same studies.

Table I summarizes the characteristics of these studies and ours. "Cooperation" in method properties means whether multiple control algorithms are coordinated when a method contains multiple algorithms. It is not the case when a single algorithm determines all task offloads or when multiple algorithms independently determine them. "Comprehensiveness" in evaluation properties means whether the methods evaluate the performance of the algorithm under practical conditions, e.g., a sufficient number of edges and clouds, and under various metrics, e.g., computation time and scalability.

Wang et al. [1] considered a cooperative three-tier computing network by leveraging vertical cooperation among devices, edge nodes, and cloud servers and horizontal cooperation between edge nodes. They also presented a parallel optimization framework by using the alternating direction method of multipliers (ADMM) method. However, they did not impose network bandwidth constraints and did not assume multi-cloud networks. They evaluated minimal conditions with four edges and 40 devices. Yuan and Zhou [2] designed a profit-maximizing collaborative computation offloading and resource-allocation algorithm to maximize system profit and guarantee task-response-time constraints. They also developed a migratory-bird optimization method that is based on simulated annealing (SA) to obtain a close-to-optimal solution. However, they did not assume multi-cloud networks and backbone-network topology. They evaluated their method under very light conditions, such as one task every 20 seconds. Kai et al. [3] developed a collaborative computing framework to process mobile devices' tasks at terminals, edge nodes, and cloud centers. They presented a pipeline-based offloading scheme, in which both mobile devices and edge nodes can offload computation-intensive tasks to an edge node and cloud center in accordance with their computation and communication capacities, respectively. However, they did not assume multi-cloud networks and backbone network topology and evaluated minimal conditions with 40 devices.

MADRL has gained attention as a solution to the problem of computation time [4], [5], [6], [7], [8]. Zhan et al. [4] designed a decentralized algorithm for computation offloading so that users can independently choose their offloading decisions. They developed their algorithm by combining MADRL and game theory. However, the actions of decentralized agents based on game theory fall into a sub-optimal solution of the Nash equilibrium. They only considered EC and did not consider CC. Nguyen et al. [5] presented a new collaborative offloading framework that is based on MADRL in heterogeneous edge networks where each ED acts as an intelligent agent to make offloading decisions collaboratively and achieve optimal system utility. However, they only considered EC and did not consider CC. Hou et al. [6] introduced a hierarchical task-offloading and resource-allocation method that is based on MADRL for the Cybertwin-based network. It can promote the flexible collaboration of EDs, EC servers, or CC servers to improve system processing efficiency and security. Although they described the formulation in detail, their paper remains only a concept, and they evaluated minimal conditions with

a single cloud, three edges, and 100–300 devices. Ding and Lin [7] considered cooperative task offloading, where all edge servers cooperate to achieve good performance for the entire edge-CC system, such as low latency and energy costs. They addressed MADRL-based task offloading that takes into account cooperation among agents. However, they evaluated minimal conditions with a single cloud and five edges. They evaluated only one snapshot and did not evaluate statistical performance. They also did not conduct any practical evaluation, e.g., computation time or scalability. Zhang et al. [8] considered a three-layer distributed multi-access edge computing network where there are multiple clouds, EC servers, and EDs. They developed a distributed scheme that is based on MADRL. Each cloud jointly determines the offloading task and resource-allocation strategy on the basis of its inference of other cloud decisions. However, they evaluated minimal conditions with 1–3 clouds, 1–3 edges, and 3–9 devices.

In summary, previous studies did not consider CC or targeted only the network with a single cloud. Zhang et al. [8] assumed multi-cloud networks, but their evaluation was under the limited condition of three edge nodes, and effectiveness of their method under practical conditions remains unclear. Previous studies did not consider the link capacity and topology of the backbone network. They assumed that the backbone-network link between clouds and edges was as if it were a single link. To the best of our knowledge, our study is the first to focus on optimal task offloading for multi-cloud and multi-edge networks considering network topology and bandwidth constraints. Several studies have addressed MADRL-based task offloading by considering cooperation among agents similar to ours but do not meet the above network requirements. Furthermore, we introduce a generalized task model representing various task types. Previous studies evaluated the performances of their methods under conditions where they generated only one type of task uniformly. We evaluated the effectiveness of the proposed method in terms of performance, network-topology dependency, computation time, scalability regarding the number of tasks, and generalization performance for unknown task patterns. Such a comprehensive evaluation had not been conducted.

III. Problem Formulation

A. Overview

We describe an overview of the task-offloading problem. We assume a network consisting of EDs, edges, and clouds. EDs generate various tasks with diverse applications. Each task consists of the required computing demand, traffic demand, and maximum permissible latency to accomplish the task. Each ED can compute its tasks locally or offload tasks to the neighboring edge or cloud. We collectively refer to edges and clouds as nodes. All nodes have computing resources to execute the tasks instead of EDs, called edge or cloud servers. All nodes also have a function of traffic forwarding to another node as a router. Only each edge has a function that determines the optimal node to offload each task. Each cloud cannot offload tasks to other nodes, only execute the tasks. We assume that the ED determines whether to offload its tasks; optimizing the ED's decision is outside the scope of this paper. We aim to optimize the offloaded server and the route between ED and server for the accepted task.

TABLE II
Notation Definitions for Physical-Network Model

Notation	Definition
$\boldsymbol{G}(\boldsymbol{N},\boldsymbol{L})$	Network graph
$n \in \boldsymbol{N}$	Node
link $(i,j) \in \boldsymbol{L}$	Link from node i to node j
$e \in \boldsymbol{E} \subset \boldsymbol{N}$	Edge
$c \in \boldsymbol{C} \subset \boldsymbol{N}$	Cloud
v_i^N	i^{th} node-processing capability
w_i^N	i^{th} node capacity
w_{ij}^L	(i,j) link capacity
α_{ij}^L	Propagation latency of link (i,j)

We describe the procedure of our task-offloading method. We consider a discrete time-step t and assume that each ED has one or more tasks and consider $\mathcal{K}$ tasks during $t \in [0, T]$. At the beginning of each t, each task arrives at the nearest edge of each ED. Each edge observes the information about tasks accepted at each edge and the overall network usage of all nodes and links. On the basis of the observation, our method deployed in each edge calculates the optimal node to offload the task (see Section V for details). The offloading node is calculated only once, and the neighboring edges that receive the tasks cannot offload it again to the other edge or cloud. When multiple tasks arrive simultaneously on each edge at t, our method repeats determining the offloading node in a first-in-first-out (FIFO) manner. The method then aggregates the traffic-demand information between nodes and calculates and updates the optimal route between nodes. Next, each edge forwards tasks to the optimal nodes through the optimal route, and the node executes the task and returns the result to the ED. The upload and download routes of each task may differ since our method handles the upload and download traffic as individual traffic demands. After a certain amount of time, it proceeds to the next $t + 1$. The executing tasks continue to consume the resources of the offloaded node and the link(s) it traverses until it returns a result to the ED. The tasks accepted at t do not need to be completed before $t + 1$. Note that it is also adequate to calculate and update the route once every several steps. In this case, our method calculates the optimal route on the basis of the average traffic volumes between nodes for several steps.

B. Network Model

Table II summarizes the notation definitions of the physical-network model. We consider the physical-network graph $G(\boldsymbol{N}, \boldsymbol{L})$ consisting of a physical node set $\boldsymbol{N}$ and physical link set $\boldsymbol{L}$. We assume that each physical node has a role as an edge or cloud. We also assume that EDs connect to the nearest edge through the access network, which is not included in $G\ (N, L)$ in this paper. We also assume that the routes in the access network take the shortest path, and route optimization in the access network is outside the scope of this paper.

TABLE III
NOTATION DEFINITIONS FOR TASK MODEL

Notation	Definition
$t \in T$	Time-step (T: Total time steps)
$\mathcal{D} := \{\boldsymbol{D}_k\}$	Task set ($k \in \mathcal{K}$, $\mathcal{K}$: Total tasks)
t_k	Acceptance time of kth task
β_k	Task type of kth task
C_k	Computing demand of kth task
B_k^{up}, B_k^{down}	Upload/Download traffic demand of kth task
$\tau_k^{\max}$	Maximum permissible latency of kth task

We denote the edge as $e \in \boldsymbol{E} \subset \boldsymbol{N}$ and the cloud as $c \in \boldsymbol{C} \subset \boldsymbol{N}$. We also denote the numbers of nodes, edges, and clouds as $|\boldsymbol{N}|$, $|\boldsymbol{E}|$, and $|\boldsymbol{C}|$, respectively. We denote the node-processing capability of the ith node as $v_i^N \in \mathbb{R}^+$, which indicates the limit of computing resources, e.g., the central processing unit (CPU) capability per second in the ith node ([G cycles/s]). Here, $\mathbb{R}^+$ indicates the set of positive real numbers. We also denote the node capacity of the ith node as $w_i^N \in \mathbb{N}$, which indicates the upper limit of the number of allocated tasks. We assign one CPU core to each task, i.e., w_i^N equals the number of CPU cores in the ith node. We denote the bandwidth capacity of link (i, j) as $w_{ij}^L \in \mathbb{R}^+$, which indicates the limit of bandwidth resources [Mbps]. All links also have propagation latency depending on the distance between each node. We define the propagation latency of link (i, j) as $\alpha_{ij}^L \in \mathbb{R}^+$ [ms]. A concrete example is shown in Fig. 1.

C. Task Model

Table III summarizes the notation definitions of the task model. We describe a task model for uniformly representing various types of tasks of EDs. We define the task set as $\mathcal{D} = \{\boldsymbol{D}_k\}$ and the kth task as $\boldsymbol{D}_k := [t_k, \beta_k, C_k, B_k^{\text{up}}, B_k^{\text{down}}, \tau_k^{\max}]$. Here, $t_k \in T$ is the accepted time of the kth task, $\beta_k \in \mathbb{N}$ is the task type uniquely given for each application, C_k is the required computing demand [G cycles], $B_k^{\text{up}} \in \mathbb{R}^+$ and $B_k^{\text{down}} \in \mathbb{R}^+$ are the required upload and download traffic demands [Mbits], and $\tau_k^{\max} \in \mathbb{R}^+$ is the maximum permissible latency to accomplish the kth task [ms]. Regarding the relationship between t_k and $\tau_k^{\max}$, if we can complete the processing of the task k by the time $t_k + \tau_k^{\max}$, we satisfy the requirements of the task k. The computing demand refers to the total amount of CPU cycles needed to accomplish the task. The traffic demand refers to the amount of traffic generated by the task. The reason for distinguishing between upload and download traffic is to accommodate tasks, the traffic volume of which changes before and after the computation process on the assigned server. When $\boldsymbol{D}_k$ is accepted, it consumes the computing and bandwidth resources on G ($\boldsymbol{N}$, $\boldsymbol{L}$) depending on the amount of computing and traffic demand of $\boldsymbol{D}_k$. If the task is assigned to the nearest edge of the ED, the bandwidth resources consumed on G ($\boldsymbol{N}$, $\boldsymbol{L}$) are regarded as 0. Note that if the task parameters are unknown, other systems need to be used to estimate the parameters, which is out of the scope of this paper.

TABLE IV
NOTATION DEFINITIONS FOR TASK-OFFLOADING PROBLEM

Notation	Definition
$\boldsymbol{X}_t := \left\{x_{ij,t}^{pq}\right\}$	Proportion of passed m_t^{pq} on link (i, j)
$\boldsymbol{Y} := \{y_{kn}\}$	Task allocation (task k, node n)
$\boldsymbol{M}_t := \left\{m_t^{pq}\right\}$	Traffic matrix from node p to node q
$\boldsymbol{Z} := \{z_{ke}\}$	Device placement (task k, edge e)
$\boldsymbol{U}_t^N := \left\{u_{i,t}^N\right\}$	i^{th} node utilization at step t
$\boldsymbol{U}_t^L := \left\{u_{ij,t}^L\right\}$	(i, j) link utilization at step t
$U_t^N = \max_i\left(\boldsymbol{U}_t^N\right)$	Maximum node utilization at step t
$U_t^L = \max_{ij}\left(\boldsymbol{U}_t^L\right)$	Maximum link utilization at step t
τ_k^N, τ_k^{RTT}	Node latency and RTT latency of kth task
λ	Weighting parameter of objective function
$\mathbb{I}_{k,t}$	Binary variable indicating executing tasks
$\mathcal{P}_k^{\text{up}}$, $\mathcal{P}_k^{\text{down}}$	Upload and download path set of kth task
τ_p	Latency coefficient of path p
$\mathcal{L}_p$	Link set along with path p
r_p	Traffic-splitting ratio of path p

D. Optimization Problem

We formulate the optimal task-offloading problem, which aims to minimize Eq. (1) while satisfying the constraints in Eqs. (2)–(14). Table IV summarizes the notation definitions of the task-offloading problem. We aim to find optimal task-allocation variables $\boldsymbol{Y}$ and routing variables $\boldsymbol{X}_t$. Here, $\boldsymbol{Y} := \{y_{kn}\}$ shows the task allocation in which y_{kn} is 1 if the computing demand of the kth task is assigned to the nth node; otherwise, 0. The $\boldsymbol{X}_t := \{x_{ij,t}^{pq}\}$ shows the proportion of traffic demand m_t^{pq} from the origin node p to destination node q passing through link (i, j) at t. Here, $\boldsymbol{M}_t := \{m_t^{pq}\}$ shows the traffic demand matrix between nodes p and q at t. The m_t^{pq} represents the total traffic demand between nodes p and q at t. The nodes p and q in m_t^{pq} are determined by device placements and task-allocated servers. A detailed formulation is shown in Eqs. (12)–(13). Since we assume multi-path routing, $x_{ij,t}^{pq}$ can take a continuous value between 0 and 1. We also define the ED placement as $\boldsymbol{Z} := \{z_{ke}\}$, in which z_{ke} is 1 if the nearest edge of the ED requested k^{th} task is the eth edge; otherwise, 0. We assume that $\boldsymbol{Z}$ is constant during $t \in [0, T]$ to ignore the effects of ED movement, e.g., handover. In other words, we assume that the ED requesting to process the task at edge e stays near edge e for the processing time, e.g., at least a few seconds.

We introduce the objective function:

$$\min \ : \ \sum_{t \in T}\left(U_t^N + U_t^L\right) + \lambda \sum_{k \in \mathcal{K}}\left(\frac{\tau_k^N + \tau_k^{\text{RTT}}}{\tau_k^{\max}}\right) / |\mathcal{K}|, \quad (1)$$

where U_t^N and U_t^L show the maximum node and link utilization at t, which are respectively defined as $U_t^N := \max_i(\boldsymbol{U}_t^N)$ and $U_t^L := \max_{ij}(\boldsymbol{U}_t^L)$. We denote the ith node utilization as $u_{i,t}^N \in \boldsymbol{U}_t^N$ and the link (i, j) utilization as $u_{ij,t}^L \in \boldsymbol{U}_t^L$. The terms τ_k^N and τ_k^{RTT} show the node and round-trip time (RTT) latency of the kth task. RTT is the time it takes to get from the source to the destination and back from the destination to the source. Their definitions and formulations are described in

the subsection on the latency constraints. The term λ indicates the weighting parameter determining the importance ratio of resource efficiency and task latency.

We impose three types of constraints: node capacity, link capacity, and task latency. We first define the binary variable $\mathbb{I}_{k,t}$ as follows:

$$\mathbb{I}_{k,t} := \begin{cases} 1 \left(t_k \leq t \leq t_k + \tau_k^N + \tau_k^{\mathrm{RTT}}\right) \\ 0 \ (\text{otherwise}), \end{cases} \tag{2}$$

where $\mathbb{I}_{k,t}$ is 1 if the kth task is executing at *t*; otherwise, 0. Here, t_k is the acceptance time of the kth task and $t_k + \tau_k^N + \tau_k^{\mathrm{RTT}}$ shows the completion time of the kth task. The total latency $\tau_k^N + \tau_k^{\mathrm{RTT}}$ of the kth task can be modeled by Eqs. (15) and (16) (details below). Latency in a real-world environment does not necessarily occur as modeled. However, in a real-world environment, we can measure latency and can use measured values instead of the modeled equations.

1) Node-Capacity Constraints: A task-allocation variable y_{kn} is formulated to minimize the maximum node utilization U_t^N while satisfying the node-capacity constraints as follows:

$$\text{s.t.}: \sum_{n \in \boldsymbol{N}} y_{kn} = 1 \ (\forall k \in \mathcal{K}) \tag{3}$$

$$\sum_{k \in \mathcal{K}} \mathbb{I}_{k,t}\, y_{kn} \leq w_n^N\, U_t^N \ (\forall n \in \boldsymbol{N}) \tag{4}$$

$$y_{kn} \in \{0,1\} (\forall k \in \mathcal{K}, \forall n \in \boldsymbol{N}) \tag{5}$$

$$0 \leq U_t^N \leq 1. \tag{6}$$

Equation (3) shows that the computing demand of each task must be allocated to any node. Equation (4) shows the constraint of node capacity. The term $\mathbb{I}_{k,t}\, y_{kn}$ in Eq. (4), the product of two binary variables, is 1 if the kth task is allocated to node *n* and is executing at *t*, because $\mathbb{I}_{k,t}\, y_{kn} = 1$ if $\mathbb{I}_{k,t} = 1$ and $y_{kn} = 1$. By calculating the sum over *k*, the left side of Eq. (4) shows the number of tasks executing at node *n* at *t*. The left side of Eq. (4) shows the nth node capacity at *t* when the maximum node utilization is U_t^N. In other words, it shows the maximum number of tasks that the nth node can perform at *t*. We impose the upper bound $w_n^N\, U_t^N$, which is stronger than w_n^N. Equations (5)–(6) show the range of variables.

2) Link-Capacity Constraints: We formulate the link-capacity constraints as a multi-commodity flow problem, which is a network flow problem with multiple commodities (i.e., traffic flow demands) between source and destination nodes. These equations give the capacity and flow conservation constraints that the routing variables must satisfy. If we find routing variables that pass through the shortest path, we can use the shortest path algorithm, such as Dijkstra or Bellman-Ford. However, these algorithms do not take capacity constraints into account. If all traffic demands pass through the shortest path, traffic may concentrate on a single link, causing congestion.

A routing variable $x_{ij,t}^{pq}$ is formulated to minimize the maximum link utilization U_t^L while satisfying the link-capacity constraints as follows:

$$\text{s.t.}: \sum_{j:(i,j) \in L} x_{ij,t}^{pq} - \sum_{j:(j,i) \in L} x_{ji,t}^{pq} = 0 \tag{7}$$
$$(\forall p, q \in \boldsymbol{N}, i \neq p, i \neq q)$$

$$\sum_{j:(i,j) \in L} x_{ij,t}^{pq} - \sum_{j:(j,i) \in L} x_{ji,t}^{pq} = 1 \tag{8}$$
$$(\forall p, q \in \boldsymbol{N}, i = p)$$

$$\sum_{p,q \in \boldsymbol{N}} m_t^{pq}\, x_{ij,t}^{pq} \leq w_{ij}^L\, U_t^L \tag{9}$$
$$(\forall (i,j) \in L, \forall p, q \in \boldsymbol{N})$$

$$0 \leq x_{ij,t}^{pq} \leq 1 \ (\forall (i,j) \in L, \forall p, q \in \boldsymbol{N}) \tag{10}$$

$$0 \leq U_t^L \leq 1. \tag{11}$$

Equations (7) and (8) show the traffic-flow conservation law. Equation (7) shows that the traffic flowing into a node equals the traffic flowing out of the node except the source node *p* and destination node *q*. Equation (8) shows that the flow out of the source node *p* is 1. The traffic-flow conservation law at the destination node *q* is guaranteed when Eqs. (7) and (8) are satisfied, which is proved in [14]. Equation (9) shows the link-capacity constraints. We impose the upper bound $w_{ij}^L U_t^L$, which is stronger than w_{ij}^L. Equations (10) and (11) show the range of variables. The term m_t^{pq} in Eq. (9) can be formulated as follows:

$$m_t^{pq} = \sum_{k \in \mathcal{K}} B_k^{\mathrm{up}}\, \mathbb{I}_{k,t}\, z_{kp}\, y_{kq} \tag{12}$$

$$m_t^{qp} = \sum_{k \in \mathcal{K}} B_k^{\mathrm{down}}\, \mathbb{I}_{k,t}\, z_{kp}\, y_{kq}$$
$$(\forall p \in \boldsymbol{E}, \forall q \in \boldsymbol{N}, p \neq q). \tag{13}$$

Equation (12) shows the upload traffic demands from origin node *p* to destination node *q*. Here, z_{kp} and y_{kq} determine the node *p* and node *q*, and $\mathbb{I}_{k,t}$ extracts the executing tasks. Equation (13) shows the download traffic demands from node *q* to node *p*, which is the opposite of the upload.

3) Latency Constraints: Task latency is the sum of all possible delays a task experiences during offloading, including processing, transmission, propagation, and queuing latency. Processing latency is the delay it takes to process the task at the edge or cloud server. We refer to this latency as node latency τ_k^N. Transmission latency is the time it takes to push all the traffic onto the link, calculated by dividing the number of bits by the transfer rate. Propagation latency is the time it takes for traffic to travel from the origin to the destination, determined by the distance between nodes and the speed of signal propagation. For a wired network, propagation latency can be assumed to depend only on distance because the speed of light is constant in fiber. Queuing latency is the time that traffic waits in the queue before it can be processed. We consider the sufficient buffer and assume that the queuing latency is zero. We refer to the sum of the transmission latency and the propagation latency as RTT latency τ_k^{RTT}.

Latency constraints of kth task are formulated as follows:

$$\text{s.t.}: \tau_k^N + \tau_k^{\mathrm{RTT}} \leq \tau_k^{\max} \ (\forall k \in \mathcal{K}). \tag{14}$$

The definitions of the node latency τ_k^N and RTT latency τ_k^{RTT} of the kth task are as follows:

$$\tau_k^N := \sum_{n \in \boldsymbol{N}} y_{kn} \frac{C_k}{v_n^N} \tag{15}$$

$$\tau_k^{\mathrm{RTT}} := \max_{p \in \mathcal{P}_k^{\mathrm{up}}} \left(\tau_p\left(B_k^{\mathrm{up}}\right)\right) + \max_{p \in \mathcal{P}_k^{\mathrm{down}}} \left(\tau_p\left(B_k^{\mathrm{up}}\right)\right). \tag{16}$$

Equation (15) shows that τ_k^N is determined by the computing-demand size of the task C_k and the node-processing capability per second v_n^N. The term C_k/v_n^N is the time it takes to process C_k at node *n*. The term y_{kn} determines the node *n* to which the task *k* is allocated, as explained in Eq. (15). Equation (16) shows that τ_k^{RTT} is determined by the bottleneck path with the maximum latency when the task goes through multiple paths. Here $\mathcal{P}_k^{\mathrm{up}}$ and $\mathcal{P}_k^{\mathrm{down}}$ are the set of upload and download paths of the kth task, which is calculated on the basis of $\boldsymbol{X}_t$, $\boldsymbol{Y}$, and $\boldsymbol{Z}$. The term $\tau_p(b)$ is the function that shows the latency when the traffic demand *b* goes through the path *p*. The $\tau_p(b)$ is formulated as follows:

$$\tau_p(b) := r_p \cdot \frac{b}{\min_{(i,j) \in \mathcal{L}_p}\left(w_{ij}^L\right)} + \sum_{(i,j) \in \mathcal{L}_p} \alpha_{ij}^L. \tag{17}$$

The first term in Eq. (17) shows the transmission latency when the traffic demand *b* goes through the path *p*. The term r_p is the traffic-splitting ratio of path *p* calculated by $\boldsymbol{X}_t$ that satisfies the following constraints: $\sum_p (r_p) = 1$. The term $\mathcal{L}_p$ is the link set along with path *p*. The term $\min_{(i,j)\ \in \mathcal{L}_p}(w_{ij}^L)$ indicates the bottleneck bandwidth on path *p*. The second term in Eq. (17) shows the propagation latency of path *p*, which is the sum of the propagation latency α_{ij}^L of the link $(i,j)\ \in \mathcal{L}_p$.

IV. Reinforcement Learning

We briefly review single-agent RL and MARL. The recent review of MARL is described in detail [15], [16], [17].

A. Single-Agent Reinforcement Learning

Single-agent RL solves the decision problem in which an agent interacts with an environment. The agent observes state $s \in \mathcal{S}$, takes action $a \in \mathcal{A}$, receives reward *r*, and transfers to the new state $s' \in \mathcal{S}$. The goal of the agent is to maximize the action value *Q*(*s*, *a*), which is defined as the expectation of the sum of rewards obtained in the future when action *a* is selected in state *s*. Q-learning [9] is a widely used RL algorithm and updates the Q-function by:

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left[r + \gamma \max_{a' \in \mathcal{A}} Q\left(s', a'\right)\right], \tag{18}$$

where α is a learning rate and γ is a discount factor.

Deep RL (DRL) [10], [18] dramatically improves the generalization and scalability of traditional RL algorithms and can handle continuous and high-dimensional state space by approximating the *Q*(*s*, *a*) with a deep neural network (DNN). Among many studies on DRL, Deep Q-network (DQN) [18] is a well-known study that has achieved human-level control through DRL and has achieved generic performance in many classic games. Our method uses DQN as the basis for the DRL algorithm because DQN has achieved stable and high performance in various use cases and is suitable for handling discrete actions. The optimal task offloading problem must handle discrete actions because it requires selecting the best server from a set of edge or cloud servers.

DQN uses a replay memory to store the transition tuple $\langle s, a, r, s' \rangle$. DNN parameters θ are learned by sampling batches *b* of transitions from the replay memory and minimizing the squared error:

$$\mathcal{L}(\theta) = \sum_{i=1}^{b}\left[\left(y_i^{\mathrm{DQN}} - Q(s,a;\theta)\right)^2\right], \tag{19}$$

$$y^{\mathrm{DQN}} = r + \gamma \max_{a' \in \mathcal{A}} Q\left(s', a'; \theta^-\right), \tag{20}$$

where θ^- are the parameters of a target network that are periodically copied from θ and kept constant for several iterations. It was reported that the performance of RL algorithms drastically worsens as the number of candidate actions increases due to the decrease in the sampling efficiency of $\langle s, a, r, s' \rangle$ and increase in the error of the DNN [10]. Therefore, stable DRL becomes difficult as the number of actions increases.

DRL performs better when historical information is used for learning. DQN defines the last four frames as the current state in the classic game environment to consider historical information. This approach was successful enough for games where reflexes are critical, but it became clear that DQN needed more than four frames for some games. To address these shortcomings, Deep Recurrent Q-Network (DRQN) [19] adds recurrency to DQN by replacing the first post-convolutional fully connected layer with a recurrent long short-term memory (LSTM). DRQN successfully integrates historical information and improves the performance of DQN in some games, even though it only sees a single frame at each time step. In addition, DRQN with the recurrent network can better adapt during an evaluation when the quality of the observations changes. Therefore, our method introduces DRQN for the DNN layer.

B. Multi-Agent Reinforcement Learning

MARL is a system of multiple agents interacting within a shared environment. Each agent works with the other agent(s) to achieve a given goal. It is used for learning a complex environment by dividing a single work into multiple sub-works. We can reduce the learning cost of each agent by assigning each agent to each work. Due to the complexities of the environments and the combinatorial nature of the problem, most MARL problems are categorized as NP-hard problems [20].

A cooperative multi-agent environment can be described as a decentralized partially observable Markov decision process (Dec-POMDP) [21] consisting of $\langle N, \mathcal{S}, \mathcal{A}, R, P, \mathcal{O}, \gamma \rangle$, in which *N* is the number of agents, $\mathcal{S}$ is state space, $\mathcal{A} = \{\mathcal{A}^1, \ldots, \mathcal{A}^N\}$ is the set of actions for all agents, *P* is the transition probability among the states, *R* is the reward function, and $\mathcal{O} = \{\mathcal{O}^1, \ldots, \mathcal{O}^N\}$ is the set of observations for all agents. If the agents can fully observe the global state, the ith agent at *t* observes the global state s_t, takes action

a_t^i ($\boldsymbol{a}_t = \{a_t^i\}$), and receives reward r_t. If the agents cannot observe the global state, each agent only accesses its own local observation o_t^i. In this case, the ith agent at t observes its local observation o_t^i, takes action a_t^i ($\boldsymbol{a}_t = \{a_t^i\}$), and receives reward r_t. Each agent has an observation-action history $h^i \in \boldsymbol{h}$, where the history indicates a series of past observations and $\boldsymbol{h}$ is the observation-action history of all agents. The joint policy π has a joint action-value function:

$$Q^{\pi}(\boldsymbol{h}_t, \boldsymbol{a}_t) = \mathbb{E}\left[\sum_{j=0}^{\infty} \gamma^j r_{t+j} \mid \boldsymbol{h}_t, \boldsymbol{a}_t\right]. \tag{21}$$

1) Independent Q-Learning: The most naive approach to solve the MARL problem is to treat each agent independently. This idea is formalized using an independent Q-learning (IQL) algorithm [22], [23], which decomposes a multi-agent problem into a collection of simultaneous single-agent problems that share the same environment. Each agent runs Q-learning [9] or DQN [18]. IQL is scalable from the viewpoint of implementation while increasing the number of agents, and each agent only needs its local history of observations during the training. In this study, IQL was trained to minimize the loss:

$$\mathcal{L}(\theta) = \sum_{i=1}^{b}\sum_{k=1}^{K}\left[\left(y_i^k - Q_k\left(h^k, a^k; \theta^k\right)\right)^2\right], \tag{22}$$

where b is the batch size of transitions sampled from the replay memory, Q_k is the kth agent's Q-value function, and y^k is the kth agent's y^{DQN}, as in Eq. (20).

2) Value Decomposition Networks: Value decomposition networks (VDNs) [24] are used to learn a joint action-value function $Q_{tot}(\boldsymbol{h}, \boldsymbol{a})$. It is assumed that the joint action-value function Q_{tot} can be additively decomposed into N Q-value functions for N agents, in which each Q-value function Q_i relies only on the local state-action history:

$$Q_{tot}(\boldsymbol{h}, \boldsymbol{a}) = \sum_{i=1}^{N} Q_i\left(h^i, a^i\right). \tag{23}$$

Each agent observes its local state, obtains the Q-values for its action, and selects an action. The sum of Q-values for the selected action of all agents then provides the total Q-value of the problem. Because each agent's DNN is updated on the basis of the total Q-value, each agent learns the optimal behavior for all agents, i.e., they learn cooperative behavior. The loss function for VDN is as follows:

$$\mathcal{L}(\theta) = \sum_{i=1}^{b}\left[\left(y_i^{tot} - Q_{tot}(\boldsymbol{h}, \boldsymbol{a}; \theta)\right)^2\right], \tag{24}$$

where b is the batch size, $y^{tot} = r + \gamma \max_{\boldsymbol{a}'} Q_{tot}(\boldsymbol{h}', \boldsymbol{a}'; \theta^-)$ and θ^- are the parameters of a target network, as in DQN.

V. Proposed Method

A. Overview

We give an overview of the proposed task-offloading method, which is based on Coop-MADRL. We aim to find the optimal task offloading that minimizes Eq. (1) while satisfying the constraints in Eqs. (2)–(14). The decision variables are the task-allocation variables $\boldsymbol{Y}$ and routing variables $\boldsymbol{X}_t$. Our method consists of two parts: Coop-MADRL and mathematical optimization. Coop-MADRL is responsible for finding the optimal $\boldsymbol{Y}$, and mathematical optimization is responsible for finding the optimal $\boldsymbol{X}_t$. Another possible way to find the solution of two variables is to solve them in sequence, such as finding the $\boldsymbol{Y}$ and then finding the $\boldsymbol{X}_t$. However, in this case, the $\boldsymbol{Y}$ is determined on the basis of a non-optimal $\boldsymbol{X}_t$, which may degrade the performance of the solution compared with that of our method. Therefore, our method jointly optimizes $\boldsymbol{Y}$ and $\boldsymbol{X}_t$. Concretely, we calculate the reward of Coop-MADRL to find the optimal $\boldsymbol{Y}$ on the basis of the $\boldsymbol{X}_t$ calculated by mathematical optimization. The concept is based on current methods [11], [25], [26]. Our method also consists of centralized training and decentralized execution. The decentralized agents continually execute the task offloading after centralized training.

TABLE V
Notation Definitions for Proposed Method

Notation	Definition
$t^{\text{sim}} \in T^{\text{sim}}$	Time-step for simulator (T^{sim}: Total time steps)
$\mathcal{G} := \{g_e\}$	Agent set ($1 \le e \le \lvert\boldsymbol{E}\rvert$)
$s_t \in \mathcal{S}$	State at step t ($\mathcal{S}$: State space)
$\mathcal{O} := \{\mathcal{O}^e\}$	Observation sets for all agents
$o_t^e \in \mathcal{O}^e$	Observation for agent g_e at step t
$\boldsymbol{o}_t := \{o_t^e\}$	All observations at step t
$\mathcal{A} := \{\mathcal{A}^e\}$	Action sets for all agents ($\mathcal{A}^e$: Action space)
$a_t^e \in \mathcal{A}^e$	Action for agent g_e at step t
$\boldsymbol{a}_t := \{a_t^e\}$	All actions at step t
r_t	Reward for agent g_e at step t
$Q_e\left(o_t^e, a_t^n\right)$	Action-value function for agent g_e
$Q_{tot}(\boldsymbol{o}_t, \boldsymbol{a}_t)$	Joint action-value function for all agent
$\mathcal{M}$	Replay memory
$h^i \in \boldsymbol{h}$	Observation-action history
$\boldsymbol{h}$	Observation-action history of all agents

Table V summarizes the notation definitions of our method. We introduce $\lvert\boldsymbol{E}\rvert$ agents equal to the number of edges and assign each agent for each edge offloading control. The eth agent $g_e \in \mathcal{G}$ learns how to optimize task offloading for the eth edge. In centralized training, each task arrives at the nearest edge at the beginning of each t. Each agent g_e observes the information o_t^e. On the basis of the observation, the g_e determines the node to offload the task as an action a_t^e. Each agent randomly determines offloading nodes in the early stages of training but can select the best node as training progresses. Our method then aggregates the traffic demands between nodes, then the mathematical-optimization solver calculates the optimal route between nodes. The solver calculates a routing variable $\boldsymbol{X}_t$ to minimize the link utilization U_t^L while satisfying the constraints in Eqs. (7)–(11). Since the routing variable $\boldsymbol{X}_t$ is a continuous value within 0–1, as shown in Eq. (11), this problem is classified as a linear programming problem. Next, each edge forwards tasks to the offloading nodes through the optimal route, and our method calculates the reward r_t. By repeating these steps, agents collect learning samples that are combinations of $\langle \boldsymbol{o}_t, \boldsymbol{a}_t, r_t \rangle$.

Algorithm 1 Centralized Training of Coop-MADRL

1: **initialize:** agent parameters, $t \leftarrow 0$
2: **while** $t \leq T$ **do**
3: generate tasks $\mathcal{D}$ for training
4: **initialize:** environment parameters
5: **for** $t^{\text{sim}} = 0, T^{\text{sim}}$ **do**
6: **for each** $g_e \in \mathcal{G}$ **do**
7: $\mathcal{D}_{e,t} \leftarrow$ select one task $(\mathcal{D}_{e,t^{\text{sim}}})$
8: $o_t^e \leftarrow$ observation $(\mathcal{D}_{e,t}, \boldsymbol{U}_t^L, \boldsymbol{U}_t^N)$
9: $a_t^e \leftarrow$ select epsilon greedy action(o_t^e)
10: update environment$(\boldsymbol{o}_t, \boldsymbol{a}_t)$ by Alg. 3
11: $r_t \leftarrow$ calculate reward by Alg. 4
12: **if** $r_t \leq -1$ **then**
13: terminate episode: $t^{\text{sim}} \leftarrow T^{\text{sim}}$
14: **if** all $\mathcal{D}_{t^{\text{sim}}}$ are allocated **then**
15: $t^{\text{sim}} \leftarrow t^{\text{sim}} + 1$
16: $t \leftarrow t + 1$
17: store episodic transition $(\boldsymbol{o}_j, \boldsymbol{a}_j, r_j), \forall j \in$ episode steps
18: train all agents $\mathcal{G}$ by random episodic transition

On the basis of these samples, the agents are trained using Coop-MADRL. In decentralized execution, the trained agents repeat the above steps except for the learning steps. Since each agent has already learned the cooperative action for all agents in centralized training, each agent can act independently in decentralized execution.

B. Modeling

We first define the variables that represent a subset of tasks as follows:

$$\mathcal{K}_t := \{k \in \mathcal{K} \mid \mathbb{I}_{k,t} = 1\} \tag{25}$$

$$\mathcal{K}_e := \{k \in \mathcal{K} \mid z_{ke} = 1\} \tag{26}$$

$$\mathcal{D}_t := \{\boldsymbol{D}_k \in \mathcal{D} \mid t_k = t\} \tag{27}$$

$$\mathcal{D}_{e,t} := \{\boldsymbol{D}_k \in \mathcal{D} \mid t_k = t, k \in \mathcal{K}_e\}. \tag{28}$$

The $\mathcal{K}_t$ indicates the index subset of tasks executed at time step *t*. The $\mathcal{K}_e$ indicates the index subset of tasks accepted at edge *e*. The $\mathcal{D}_t$ indicates the subset of tasks accepted at *t*. The $\mathcal{D}_{e,t}$ indicates the subset of tasks accepted on edge *e* at *t*, i.e., $\mathcal{D}_{e,t} \subset \mathcal{D}_t \subset \mathcal{D}$.

A state is defined as $s_t = [\mathcal{D}_t, \boldsymbol{U}_t^L, \boldsymbol{U}_t^S]$. An observation for agent g_e is defined as $o_t^e = [\mathcal{D}_{e,t}, \boldsymbol{U}_t^L, \boldsymbol{U}_t^N]$. The candidate action set $\mathcal{A}^e$ is defined as the set of nodes that offload tasks. The node set consists of the nearest edge, neighboring edges, and neighboring clouds. The neighboring edges and clouds are one or more nodes determined by the physical-network graph $G(\boldsymbol{N}, \boldsymbol{L})$. We exclude nodes from $\mathcal{A}^e$ that have no remaining resources and edges that are more than two hops away from edge *e*. When edge *e* does not receive any tasks at *t*, agent g_e chooses the action "do nothing." We design the reward function to return a negative value if the constraints are not satisfied; otherwise, a positive value depends on the objective-function value (see Section V-E for details).

C. Formulation

Algorithm 1 shows the centralized training using Coop-MADRL. Line 1 shows the initialization of agent parameters.

Algorithm 2 Decentralized Execution of Coop-MADRL

1: $t^{\text{sim}} \leftarrow 0$, $Q_e(o^e, a^e) \leftarrow$ train all agents $\mathcal{G}$ by Alg. 1
2: **while** all tasks are allocated **do**
3: **for each** $g_e \in \mathcal{G}$ **do**
4: $\mathcal{D}_{e,t} \leftarrow$ select one task $(\mathcal{D}_{e,t^{\text{sim}}})$
5: $o_t^e \leftarrow$ observation $(\mathcal{D}_{e,t}, \boldsymbol{U}_t^L, \boldsymbol{U}_t^N)$
6: $a_t^e \leftarrow \arg\max_{a' \in \mathcal{A}^e} Q_e\left(o_t^e, a'\right)$
7: update environment$(\boldsymbol{o}_t, \boldsymbol{a}_t)$ by Alg. 3
8: **if** all $\mathcal{D}_{t^{\text{sim}}}$ are allocated **then**
9: $t^{\text{sim}} \leftarrow t^{\text{sim}} + 1$

A series of procedures (lines 2–18) is repeatedly executed until learning is complete. Lines 3–4 show the generation of tasks and initialization of environment parameters. A series of actions is called an episode, and each episode (lines 5–16) is repeatedly executed. In each episode, agents collect learning samples that are combinations of $\langle \boldsymbol{o}_t, \boldsymbol{a}_t, r_t \rangle$. We denote a time step for the network simulator as $t^{\text{sim}} \in T^{\text{sim}}$, which is reset at the beginning of each episode. In line 7, when edge *e* accepts multiple tasks at t^{sim}, agent g_e selects one task in an FIFO manner. The two task subsets can be written as the following relationship: $\mathcal{D}_{e,t} \subseteq \mathcal{D}_{e,t^{\text{sim}}} (\subset \mathcal{D})$. In line 9, a random action is selected with probability ε; otherwise, an action that maximizes $Q_e(o_t^e, a')$ is selected with probability $1 - \varepsilon$. This is to avoid convergence to a local optimum solution. Each agent executes lines 7–9 in parallel. In line 10, task offloading is updated in accordance with $\boldsymbol{a}_t$ by Algorithm 3. Line 11 shows the reward calculation. Lines 12–13 mean the termination condition of agent learning. In this algorithm, $r_t \leq -1$ is the terminate condition, i.e., the state that does not satisfy at least one constraint. In lines 14–15, if all tasks accepted at t^{sim} are allocated, it proceeds to the next $t^{\text{sim}} + 1$. Line 17 shows stores in replay memory $\mathcal{M}$. The reason for storing the samples once in replay memory is to eliminate the time dependence of collecting training samples [18]. Lines 3–17 can be paralleled because the samples of the episodic transitions stored in replay memory are independent of the storing order. In line 18, all agents $\mathcal{G}$ are trained by the history of episodic transition, which is randomly taken from $\mathcal{M}$. We used VDN [24] as the learning algorithm for all agents (see Section IV-B2 for details). To achieve global optimization, all agents learn to maximize a shared objective function, namely the joint action-value function $Q_{tot}(\boldsymbol{o}_t, \boldsymbol{a}_t)$, which helps prevent competition among the agents. In our architecture, each edge has a replay memory $\mathcal{M}$. Since the observation o_t^e of the edge *e* is composed of local task information and shared network information, episodic transitions can be stored at each edge without aggregating data from other agents.

Algorithm 2 shows the Coop-MADRL algorithm of the proposed task-offloading method. Line 1 shows the pre-training of $\mathcal{G}$ by using Algorithm 1. Next, this algorithm continually repeats lines 2–9 as long as our method accepts new tasks. In line 6, each agent selects an a_t^e that maximizes $Q_e(o_t^e, a')$.

D. Update Environment

Algorithm 3 shows the procedure of the update environment. The task-allocation variables $\boldsymbol{Y}$ and routing variables $\boldsymbol{X}_t$ are

Algorithm 3 Update Environment

1: $\boldsymbol{Y} \leftarrow$ allocate tasks$(\boldsymbol{a}_t, \mathcal{D}_t)$
2: $\boldsymbol{U}_t^N \leftarrow$ calculate node utilization$(\mathcal{D}_t, \boldsymbol{Y})$
3: $\boldsymbol{M}_t \leftarrow$ calculate traffic matrix$(\mathcal{D}_t, \boldsymbol{Y}, \boldsymbol{Z})$
4: $\boldsymbol{U}_t^L, \boldsymbol{X}_t \leftarrow$ calculate link utilization$(\boldsymbol{M}_t)$
5: $\tau_k^N, \tau_k^{\mathrm{RTT}} \leftarrow$ calculate latency$(\boldsymbol{X}_t, \boldsymbol{Y}, \boldsymbol{Z})$
6: **return** $\boldsymbol{U}_t^N, \boldsymbol{U}_t^L, \tau_k^N, \tau_k^{\mathrm{RTT}}$

Algorithm 4 Reward Calculation

1: $r_t \leftarrow \mathrm{Eff}\left(U_{t+1}^L\right) + \mathrm{Eff}\left(U_{t+1}^N\right) + \mathrm{Eff}\left(\tau_t^{\mathrm{ave}}\right)$
2: **if** $U_{t+1}^L > 1$ or $U_{t+1}^L > 1$ **then**
3: **return** -3
4: **return** $\max(-3, \min(3, r_t))$

updated in Algorithm 3. Line 1 shows the calculation of $\boldsymbol{Y}$, line 2 the calculation of $\boldsymbol{U}_t^N$, line 3 the calculation of $\boldsymbol{M}_t$ using Eqs. (12) and (13), and line 4 the calculation of $\boldsymbol{U}_t^N$. In line 4, we solve the route-optimization problem, which calculates $\boldsymbol{X}_t$ using Eqs. (7)–(11). Line 5 shows the calculation of latency. Finally, Algorithm 3 returns variables for the reward calculation.

E. Reward Calculation

We design the reward function on the basis of the objective function in Eq. (1). Algorithm 4 shows the procedure of the reward calculation for $\mathcal{G}$. The term $\mathrm{Eff}(x)$ shows the efficiency function and is defined as follows:

$$\mathrm{Eff}(x) = \begin{cases} -x+1 & (x \le 1) \\ -2x & (1 < x). \end{cases} \tag{29}$$

We design this function so that the efficiency decreases as x increases. It returns a positive value depending on x if $x < 1$; otherwise, it returns a negative value. The decrease in efficiency doubles when $x > 1$. Note that this design is essentially independent of the effectiveness of the proposed method. The term τ_t^{ave} indicates the average task latency efficiency at t, which shows the average satisfaction of latency and is defined as follows:

$$\tau_t^{\mathrm{ave}} = \sum_{k \in \mathcal{K}_t} \left(\frac{\tau_k^N + \tau_k^{\mathrm{RTT}}}{\tau_k^{\max}} \right) / |\mathcal{K}_t|. \tag{30}$$

The terms τ_k^N and τ_k^{RTT} are calculated using Eqs. (15)–(16). If it does not satisfy $U_{t+1}^L > 1$ or $U_{t+1}^L > 1$, it returns -3. Finally, it returns a clipped reward within $-3 \le r_t \le 3$. The reason for clipping the reward is to increase the stability of agent learning [18].

VI. Evaluation

We evaluated the effectiveness of the proposed method through simulations in terms of performance, network topology dependency, computation time, and scalability regarding the number of tasks. We also evaluated its generalization performance for unknown task patterns. We prepared five comparison methods and four network topologies for the evaluation. For each agent's DNN layer, we introduced DRQN [19]. We used a three-layer DNN consisting of two fully connected layers and the gated recurrent unit (GRU) layer [27]. We used Adam [28] to optimize all DNNs and set the number of neurons in the hidden layers to 64 for all DNNs. We used Double-DQN [10] as the DRL algorithm. We implemented the DRL-algorithm-based PyTorch [29], PyMARL [30], and PyMARL2 [31]. We solved the route optimization using the GNU Linear Programming Kit (GLPK) [32]. For the hyperparameters of DRL, the learning rate was set to $\alpha = 0.001$, and the discount factor was set to $\gamma = 0.99$. The parameter ε, which determines the probability of random actions, was linearly decreased from 1.0 to 0.05 over the first 10^5 steps and then fixed at 0.05. We accelerated the centralized training with parallel counts of 8.

A. Evaluation Conditions

We set the number of tasks to $\mathcal{K} = 1000$, total time steps to $T = 5.0 \times 10^5$, total time steps of each episode to $T^{\mathrm{sim}} = 50$, and weighting parameter to $\lambda = 1$. We assume that each ED has one task during $t^{\mathrm{sim}} \in [1, T^{\mathrm{sim}}]$, the acceptance time t_k is randomly generated within 1–50, and the ED placement $\boldsymbol{Z}$ is randomly generated. We set the time-step interval from t to $t+1$ on the network simulator to 100 ms. Since this interval is merely a parameter of the network simulator, the actual training time per step need not be less than 100 ms. However, the execution time of real-world task offloading should be less than 100 ms. Our proposed method satisfies these time constraints (see Section VI-F for details).

For task generation, we prepare four types of tasks assuming various use cases. We use a generalized task model to reveal that the proposed method can effectively allocate tasks even when many types of tasks are mixed. Table VI summarizes the parameters for task generation. The task model is parameterized by upload and download traffic demand, B_k^{up} and B_k^{down}, computation demand C_k, and maximum permissible latency to accomplish the task $\tau_k^{\max}$. We determine the maximum permissible latency on the basis of the upload and download traffic and computation demand. We model Type 1 tasks as basic tasks and model other types of tasks by modifying the parameters of basic tasks. Each task parameter randomly takes a continuous value within a specified range. The task parameters are reset at the beginning of each episode.

For the physical network, we prepare four network topologies. We first prepare the 12-node topology on the basis of Internet2 [33], which consists of nine edges and three clouds, as shown in Fig. 1. We connect each cloud to one randomly selected edge and fix the cloud placements for all evaluations. The values in this figure show the pair of node-processing capability and node capacity (w_i^N, v_i^N) and the pair of link capacity and propagation latency $(w_{ij}^L, \alpha_{ij}^L)$. All evaluations except for the network-topology dependency evaluation were based on this topology. We then prepare the other topologies that refer to SNDLib [34], a library of test instances of survivable fixed telecommunication network design. Table VII summarizes the network-topology parameters. For each network topology, we define neighboring edges and clouds for each edge. We define the edges with a hop

TABLE VI
TASK-GENERATION PARAMETERS

Parameter	Type 1: Basic Task	Type 2: Download-heavy Task	Type 3: Computing-heavy Task	Type 4: Latency-sensitive Task
B_k^{up} [Mbits]	0.1–50	0.1–1	90–100	0.1–10
B_k^{down} [Mbits]	0.1–50	90–100	0.1–1	0.1–10
C_k [G cycles]	0–1	0–1	10–20	0–0.1
$\tau_k^{\max}$ [ms]	$5(B_k^{\text{up}} + B_k^{\text{down}}) + 500C_k$ 1–1000	$5(B_k^{\text{up}} + B_k^{\text{down}}) + 500C_k$ 450.5–1005	$5(B_k^{\text{up}} + B_k^{\text{down}}) + 250C_k$ 2950.5–5505	$2.5(B_k^{\text{up}} + B_k^{\text{down}}) + 250C_k$ 0.5–75
Ratio	40%	20%	20%	20%

TABLE VII
NETWORK-TOPOLOGY PARAMETERS

Parameter	Notation	Internet2	Abilene	Atlanta	Geant
# of Edges	$\lvert \boldsymbol{N} \rvert$	9	12	15	22
# of Clouds	$\lvert \boldsymbol{C} \rvert$	3	3	3	3
# of Links	$\lvert \boldsymbol{L} \rvert$	16	18	25	39
Edge-processing capability	v_i^E [G cycles/s]	2	2	2	2
Cloud-processing capability	v_i^C [G cycles/s]	10	10	10	10
Edge capacity	w_i^E	20	20	20	20
Cloud capacity	w_i^C	100	100	100	100
Edge–Edge link capacity	w_{ij}^L [Gbps]	1	1	1	1
Edge–Cloud link capacity	w_{ij}^L [Gbps]	3	3	3	3
Edge–Edge propagation latency	α_{ij}^L [ms]	1	1	1	1
Edge–Cloud propagation latency	α_{ij}^L [ms]	0.5	0.5	0.5	0.5

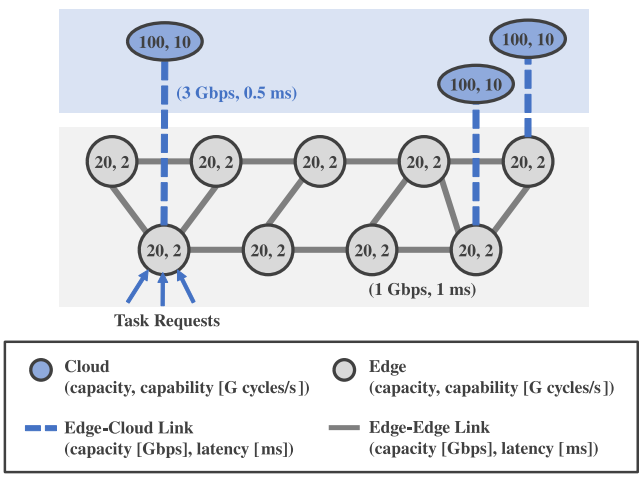

Fig. 1. Network topology (Internet2).

count of 1 as the neighboring edges for each edge. We also define the cloud(s) other than the farthest cloud from each edge as neighboring clouds for each edge.

We perform training and evaluation steps on the prepared tasks and physical networks. We run Algorithm 1 for training and Algorithm 2 for evaluation. Tasks in training and evaluation are generated under the same conditions. Note that the tasks generated during training and evaluation are different because some parameters are randomly generated. In the evaluation, the MADRL-based methods select the best action (i.e., the offloading server) on the basis of the trained agents in line 6 of Algorithm 2. The non-DRL-based methods select the offloading server on the basis of their respective algorithms, rather than selecting actions by agents.

TABLE VIII
SUMMARY OF OUR METHOD AND COMPARISON METHODS

Method	Methodology	RL-based	Cooperation
VDN (Ours)	Coop-MADRL	✓	✓
IQL	MADRL	✓	-
RA	Random Algorithm	-	-
GA	Greedy Algorithm	-	-
HA	Heuristic Algorithm	-	-
ES	Exhaustive Search	-	-

B. Comparison Methods

Table VIII summarizes our method and the comparison methods. VDN indicates the proposed Coop-MADRL-based method. IQL indicates the MADRL-based method without cooperation, i.e., each agent learns on the basis of their reward. As mentioned in Section II, there is no method for considering multi-cloud networks and network topology, but IQL corresponds to such a method [4] when it considers them. Note that we did not evaluate single-agent DRL because learning will clearly be unsuccessful due to requirements for huge training iterations until the Q-values for all actions are sufficiently close to the optimal.

We also compared our method with three algorithms that may be able to offload tasks in a computation time close to RL-based methods. The random algorithm (RA) has a policy that allocates each task to a node randomly chosen from all

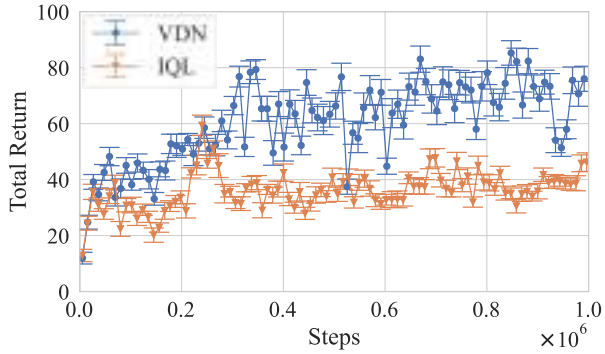

Fig. 2. Training curves tracking agent's total return in **Internet2** topology.

candidate nodes at each t. The greedy algorithm (GA) has a policy that allocates tasks to the nearest edge until the node utilization of the nearest edge exceeds a threshold and allocates tasks to the neighboring cloud after exceeding that threshold. We selected the best threshold with the best performance after evaluating all patterns in increments of 0.1 through a preliminary experiment. The GA threshold was set to 0.2, 0.2, 0.2, and 0.1 for Internet2, Abilene, Atlanta, and Geant, respectively. The heuristic algorithm (HA) has a policy that allocates tasks to suitable nodes depending on the characteristics of the task type. For this evaluation, HA allocated tasks to random nodes when the task type was 1 or 2, to the neighboring clouds when the task type was 3, and to the nearest edge when the task type was 4. This is based on the empirical rule that allocating computing-heavy tasks to the clouds and latency-sensitive tasks to the nearest edge are suitable. Note that these three algorithms also optimize routes calculated with the route-optimization algorithm at each t.

We also evaluated exhaustive search (ES), which has a policy that finds the best offloading node by exhaustively calculating each reward for all candidate nodes at t, equivalent to solving an online optimization problem every t. Note that the solution of ES is different from the exact optimal solution. Whereas the optimal solution is defined as the solution that maximizes the sum of the rewards at each time, as shown in Eq. (1), ES aims to maximize the immediate reward obtained in the present. ES also independently calculates the optimal offload node for each edge at t on the basis of the information obtained at $t-1$, which may concentrate the tasks on certain nodes.

C. Training Curve

Figure 2 shows the training curves tracking the agent's total return of VDN and IQL under the Internet2 topology. The total return is defined as the sum of rewards at each time until the end of the episode. For this evaluation, the total time step was set to $T = 1.0 \times 10^6$ to ensure that the agent's learning has converged sufficiently. We conducted five trials for every 1.0×10^4 steps with random initial conditions. The width of each bar indicates the standard deviation $(\pm\sigma)$.

The results indicate that the average total return of the two MADRL methods increased as the training progress and converged around $T = 3.0 \times 10^5$. The results also indicate that the final return of VDN was higher than IQL, which means that IQL cannot learn the suitable task offloading that maximizes the objective function while satisfying constraints. We discuss the performance details in Section VI-D.

D. Performance

1) Results: Figure 3 shows the average performance of each method. We carried out 20 calculations with random initial conditions and set the same random seeds for all trials. The width of each bar indicates the standard deviation $(\pm\sigma)$. We can roughly compare the performance of each method with the average reward. We also investigated the performance details in terms of the second to fifth metrics in Fig. 3. These metrics are the average maximum node utilization U_t^N, average maximum link utilization U_t^L, average task-latency efficiency τ_t^{ave}, and average constraint violation. The higher reward means better performance, whereas the lower values for other indicators mean better performance. When all constraints are satisfied, the node utilization, link utilization, and task latency take a value within 0–1. The constraint violation denotes the total number of times each method violates either node utilization, link utilization, or task-latency constraints.

Figure 3 shows that the proposed method (VDN) performed comparably to ES and outperformed the other methods. These results indicate that VDN and ES can maximize network-utilization efficiency and minimize task latency while reducing constraint violations. Note that constraint violations occurred for all methods because we evaluated performance under severe conditions that put heavy loads on the network. Although ES performed the best, it is problematic in terms of computation time (see Section VI-F for details).

Next, we discuss the performance comparison between VDN and other methods. We first compared VDN and IQL. Figure 3 shows that VDN performed better than IQL for all metrics. This indicates that the coordination among agents with VDN improves the performance of task offloading and can prevent constraint violations. Since each agent of IQL acts independently, task offloading is concentrated on lightly loaded resources in some cases, causing constraint violations as described in the Introduction.

We next compared VDN and the three non-RL-based computationally lightweight methods: RA, GA, and HA. The GA threshold was set to 0.2 because the best performance was obtained in the preliminary experiment. Figure 3 shows that VDN outperformed the three methods in terms of average reward. However, VDN had a higher maximum node utilization than GA and HA and higher maximum link utilization than RA. The reason is that agents choose the action that maximizes the average reward even if the node and link utilization are increased. As a characteristic of each method, GA performed comparably to or better than VDN in terms of maximum node utilization, maximum link utilization, and task latency efficiency by setting the best threshold in the preliminary experiment. However, GA had more constraint violations because it could not dynamically select allocations on the basis of the situation, resulting in a lower average reward than VDN. In addition, the maximum node utilization and task latency efficiency of HA were competitive with those of the other

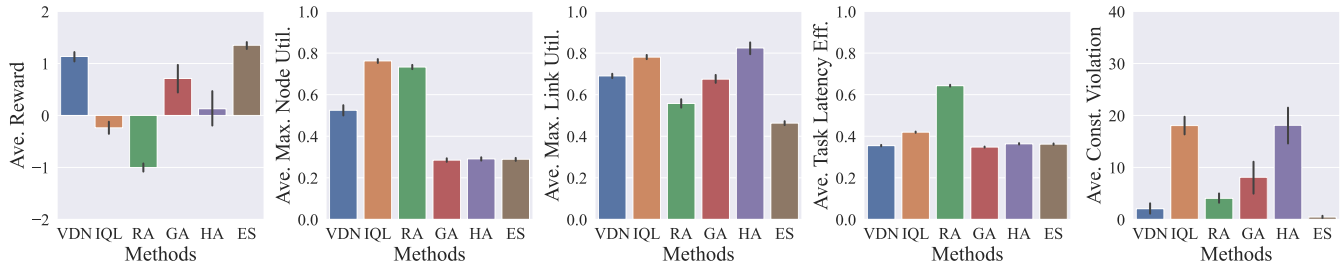

Fig. 3. Performance in **Internet2** topology.

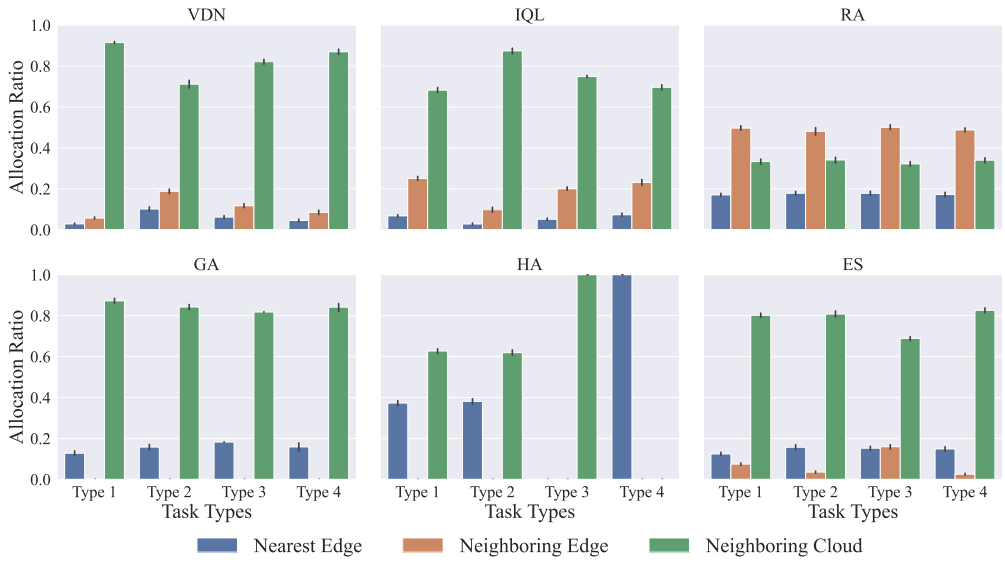

Fig. 4. Allocation ratio of each task type in **Internet2** topology.

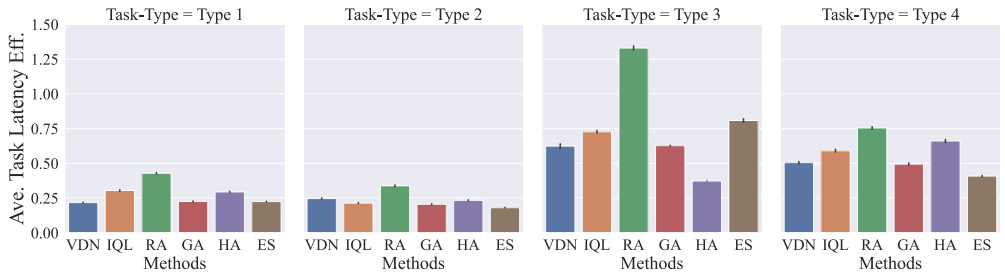

Fig. 5. Task latency of each task type in **Internet2** topology.

methods because HA designs the appropriate offloading node depending on the characteristics of each task. However, the other indicators of HA showed the worst performance because HA does not take into account the link constraints.

2) Analysis: We discuss the reasons for the performance shown in Fig. 3 by analyzing the latency and allocation ratio of each task type. Figures 4 and 5 show the latency and allocation ratio of each task type in the evaluation in Fig. 3. Here, the allocation ratio indicates the proportion of where tasks allocated to the nearest edge, neighboring edges, or neighboring clouds. The allocation ratios for VDN, IQL, and ES are determined by the task and network conditions. In contrast, the allocation ratios for RA and HA are fixed, regardless of the task and network conditions, and are determined by the algorithm design. The allocation ratio for GA is determined by the GA threshold value, which was set to 0.2 in this evaluation.

Figure 4 shows that VDN, IQL, GA, and ES primarily allocated tasks to the neighboring cloud. The result shows that the neighboring cloud is the preferred node for allocating tasks. This is because of the sufficient cloud-processing capacity and the short transmission delay between the edge and cloud, making assigning tasks to the cloud preferable in this evaluation condition. However, this result is only an example of an evaluation and does not deny the effectiveness of EC. The allocation to the edge may become dominant depending on the evaluation conditions.

Figure 4 shows that VDN changed the allocation ratio in accordance with the task type. VDN mainly allocated all types of tasks to the neighboring cloud. Looking at the ratio using the edges, VDN allocated most download-heavy tasks with high transmission costs (Task 2) to the edges. However, it allocated more computing-heavy tasks (Task 3) to

TABLE IX
AVERAGE REWARD OF EACH METHOD FOR VARIOUS NETWORK TOPOLOGIES

Method	Internet2	Abilene	Atlanta	Geant
VDN	1.13 ± 0.21	0.64 ± 0.39	1.23 ± 0.15	1.32 ± 0.18
RA	−1.00 ± 0.18	−0.79 ± 0.30	−0.53 ± 0.22	−0.34 ± 0.15
GA	0.71 ± 0.61	−0.83 ± 0.51	0.61 ± 0.49	0.89 ± 0.34
HA	0.13 ± 0.76	−1.83 ± 0.21	−0.15 ± 0.63	−0.06 ± 0.55
ES	1.35 ± 0.15	0.92 ± 0.28	1.35 ± 0.23	1.53 ± 0.18

neighboring clouds, even though the average traffic volumes of Tasks 2 and 3 were equal. We conclude that VDN learns a superior policy through cooperative learning. However, the latency-sensitive tasks (Task 4) were primarily allocated to neighboring clouds, even though they should be allocated to the nearest edge to minimize latency. We assume the reason is that VDN attempts to maximize the overall network efficiency by allocating lightweight tasks to the cloud because of limited resources at the nearest edge.

Figure 4 shows that IQL changed the allocation ratio in accordance with the task type, although it is not associated with satisfactory performance, as shown in Figure 3. RA and GA did not change the allocation ratio in accordance with the task type, which is assumed to be one reason for the low performance of these methods. Figures 4 and 5 show that HA was suitable for latency for Tasks 3 and 4 because it changes the allocation node depending on the task type. However, as mentioned in Section VI-D1, HA performed poorly because it does not take into account the link constraints.

Figure 4 shows that ES can use the nearest edges compared with VDN, which is assumed to be one reason that ES performed better than VDN. Even though ES selects the node that maximizes the reward for all candidate nodes, the performance difference between VDN and ES is slight. This is because ES aims to maximize the immediate reward obtained in the present. That is, selecting the node that maximizes the immediate reward in the present may not maximize the expectation of the sum of rewards obtained in the future.

E. Network-Topology Dependency

Table IX shows the average reward of each method for various network topologies. We carried out 20 calculations with random initial conditions and calculated the mean value and standard deviation of rewards. We did not evaluate IQL because it is evident that learning agents is more difficult in larger topologies. The results indicate that VDN outperformed the other methods except for ES in all network topologies. Thus, the proposed method can be applied to larger network topologies.

We first discuss the average performance with each topology. Table IX shows that the average performance of all methods in Abilene decreased compared with that of other topologies. Comparing the conditions between Internet2 and other topologies, as shown in Table VII, only the network size is larger, and the other parameters are the same. We consider that the graph structure of the topology determines the performance.

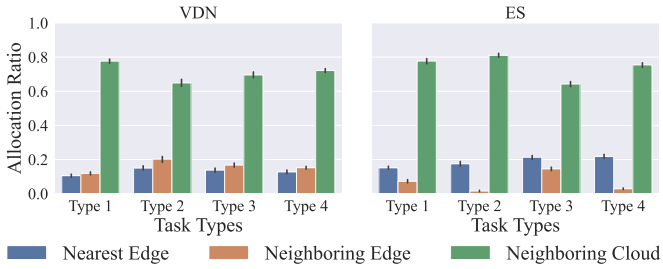

Fig. 6. Allocation ratio of each task type in **Abilene** topology.

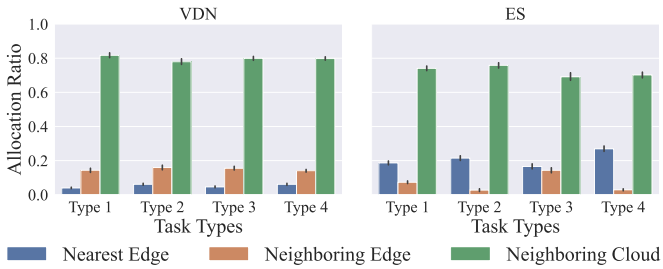

Fig. 7. Allocation ratio of each task type in **Atlanta** topology.

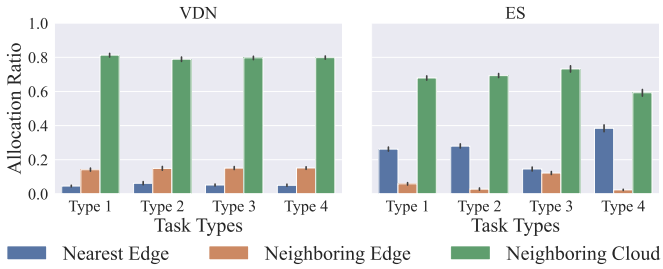

Fig. 8. Allocation ratio of each task type in **Geant** topology.

We next discuss the performance of each method. Table IX shows that VDN performed comparably to ES in all topologies, whereas the performance difference between VDN and ES increased in Abilene. Since all methods performed worse in Abilene, the performance difference between VDN and ES could be significant in environments where agent learning is complex. Table IX also shows that the performance trends of GA and HA remain unchanged. On the other hand, the performance of RA improved as the topology size increased. This is because a larger topology provides more control options, enables load balancing, and improves performance regardless of the control method.

Figures 6–8 show the allocation ratio of each task type for larger topologies. We omit the results of RA, GA, and HA because these methods do not change the allocation ratio depending on evaluation conditions. These figures show that VDN mainly allocated all types of tasks to the neighboring cloud, similar to that of Internet2. On the other hand, the allocation ratio of ES depended on network topology.

Figure 6 shows that the allocation ratio of VDN in Abilene was about the same as that in Internet2, while it increased the task allocation to the nearest edge. This means that VDN changed the allocation ratio in accordance with Abilene and

TABLE X
EXECUTION TIMES FOR VARIOUS NETWORK TOPOLOGIES

Method	Internet2	Abilene	Atlanta	Geant
VDN (Ours)	0.19 ms	0.26 ms	0.26 ms	0.25 ms
RA	0.07 ms	0.08 ms	0.09 ms	0.10 ms
GA	0.08 ms	0.12 ms	0.12 ms	0.11 ms
HA	0.11 ms	0.23 ms	0.28 ms	0.45 ms
ES	0.89 s	2.5 s	6.0 s	32.4 s

TABLE XI
TRAINING TIMES FOR VARIOUS NETWORK TOPOLOGIES

Method	Internet2	Abilene	Atlanta	Geant
VDN (Ours)	0.3 days	0.6 days	1.7 days	5.8 days

learned the superior policy in accordance with the environment. Figures 7–8 show that the allocation ratio in larger topologies was similar to that of Internet2. However, the difference in the allocation ratio for each task type became smaller as the topology size increased. This means that learning in accordance with task type becomes more difficult as the topology size increases.

In summary, VDN outperformed the other methods except for ES in all network topologies. In addition, VDN can allocate tasks in accordance with the task type when the topology is small, but this becomes more difficult as the topology size increases. Although VDN cannot learn by task type in larger topologies, it achieves superior performance competitive with that of ES by learning the appropriate allocation to edges and clouds depending on the situation.

F. Computation Time

Table X shows the average execution time of each method for various network topologies. The execution time denotes the computation time required to determine the allocated node for a single task, which corresponds to line 4 in Algorithm 2 for VDN. We used Apple M1 Ultra for the evaluation. The execution time of all methods except for ES was less than 1 ms for all topologies. The execution time tended to increase as topology size increases, but the increase is almost negligible for task latency. This means that VDN performs sufficiently in terms of computation time. However, the execution time of ES was about 1 s in the fastest case. The execution time of these task-offloading methods was added to the task latency in real-world scenarios. When the execution time takes 1 s, the task latency cannot be shorter than 1 s. Therefore, it is a critical for many tasks to keep latency to less than a few tens of milliseconds. The execution time drastically increased as the topology size increased because the computational complexity of the optimization calculation also drastically increased. We conclude that VDN is the best task-offloading method, considering the allocation performance shown in Table IX and execution times shown in Table X.

Table XI shows the training time of VDN for various network topologies. The training time denotes the computation time required for the agents to complete the training of all time-steps $T = 5 \times 10^5$. The training time increases as topology size increases. Most of the increase in the training time was due to the increased computational complexity of the network simulation, which corresponds to Algorithm 3. Although larger topologies require longer training times, this is not problematic since the training only needs to be done once.

G. Latency-Weighting-Parameter Dependency

Table XII shows the weighting-parameter dependency of the objective function shown in Eq. (1). The weighting parameter λ determines the importance ratio of resource efficiency and task latency. We carried out 20 calculations with random initial conditions and set the same random seeds for all calculations. The results indicate that, as λ increased, VDN improved in task-latency efficiency and worsened the sum of node and link utilization. We revealed that the proposed method can adjust the importance of the objective function by λ. However, these indicators show no drastic change for the increase in λ. This is because reducing task latency leads to better node and link utilization. For example, to reduce task latency, the task-offloading method must balance server loads between clouds and edges in accordance with the task characteristics, which improves node utilization. To reduce task latency, it must minimize the route length of the task, which decreases link loads and improves link utilization.

H. Scalability Regarding Number of Tasks

We evaluated the scalability of VDN regarding the number of tasks $\mathcal{K}$. We simultaneously evaluated the generalization performance of VDN regarding the number of tasks. The generalization performance measures how accurately a method can perform for previously unseen data. In other words, we evaluated the performance of VDN with a different number of tasks than during training. We increased link capacity and node capacity depending on $\mathcal{K}$, as shown in Table XIII. Therefore, the relative load on the system is kept constant and we can only evaluate the scalability of VDN. Other parameters were those in Table VII. We evaluated performance under strict conditions where the capacity increase is 95% of the constant multiple.

Table XIV shows the scalability of VDN regarding the number of tasks $\mathcal{K}$ in Internet2. We carried out 20 calculations with random initial conditions and calculated the average and the standard deviation of rewards. We use agents trained when $\mathcal{K} = 1000$ for all conditions. VDN kept the average rewards even as the number of tasks increased. The agents of VDN learn the policy from accepted task information and each node and link utilization information. Since this information of the physical network is normalized and independent of the number of tasks, VDN can determine the preferred task allocation without depending on the number of tasks. We conclude that the proposed method has scalability and generalization performance regarding the number of tasks. However, the variance of rewards tends to increase as the number of tasks increases. Therefore, retraining the agents is advisable when the number of tasks significantly differs from the training condition.

I. Generalization Performance for Task Types

We evaluate the generalization performance of VDN for various task types. We evaluate the average reward when each

TABLE XII
LATENCY-WEIGHTING-PARAMETER DEPENDENCY

Case	Ave. Max. Node Util.	Ave. Max. Link Util.	Ave. Task Latency Eff.	Constraint Violation
$\lambda = 1$	0.52 ± 0.06	0.69 ± 0.02	0.35 ± 0.00	2.1 ± 2.5
$\lambda = 2$	0.55 ± 0.07	0.71 ± 0.02	0.34 ± 0.00	2.8 ± 2.4
$\lambda = 3$	0.34 ± 0.04	0.74 ± 0.03	0.33 ± 0.00	3.8 ± 3.2

TABLE XIII
NETWORK PARAMETERS FOR SCALABILITY EVALUATION

Case	w_i^E	w_i^C	w_{ij}^L	w_{ij}^L
$\mathcal{K} = 1000$	20	100	1	3
$\mathcal{K} = 2000$	38	190	1.9	5.7
$\mathcal{K} = 5000$	95	475	4.75	14.25
$\mathcal{K} = 10000$	180	900	9	27

TABLE XIV
SCALABILITY EVALUATION REGARDING NUMBER OF TASKS

Case	Ave. Reward
$\mathcal{K} = 1000$	1.13 ± 0.21
$\mathcal{K} = 2000$	1.36 ± 0.25
$\mathcal{K} = 5000$	1.25 ± 0.60
$\mathcal{K} = 10000$	1.19 ± 0.61

TABLE XV
VARIOUS CASES FOR GENERALIZATION-PERFORMANCE EVALUATION

Case	Ave. Reward
$\mathcal{K} = 1000$	1.13 ± 0.21
$\mathcal{K} = 2000$	1.36 ± 0.25
$\mathcal{K} = 5000$	1.25 ± 0.60
$\mathcal{K} = 10000$	1.19 ± 0.61

task type ratio differed from the training condition. Table XV shows various cases with different ratios of task types. We set the conditions in Table VI as default. We prepared cases that increase the ratio of each task type and decrease the ratio of basic tasks. We also prepared cases that change the ratio of three task types per 5% and the ratio of basic tasks per 15%.

Table XVI shows the average reward and other indicators for various cases changing the ratio of task types. We carried out 20 calculations with random initial conditions and set the same random seeds for all calculations. The average reward decreased as the ratio of Tasks 2–4 increased from the training condition. For Task 3, VDN had generalization performance until the task ratio increased by 2%. The average reward notably decreased for increases of 5%. This is because tasks with high average computing demand occupy the computing resources over a long period and reduce node utilization. For Tasks 2 and 4, VDN had generalization performance even when the task ratio increased by 20%. This is because Tasks 2 and 4 occupy computing resources for only a short time. Similarly, when we simultaneously increased the ratio of the three task types by 5%, VDN showed no generalization performance. In other words, the proposed method has generalization performance when the prediction error in the proportion of computing-heavy tasks is within 2%. Conversely, for the case in which we simultaneously decreased the ratio of the three task types, the performance of VDN improved. Therefore, we conclude that the proposed method can have generalization performance for various task types by pre-training with the predicted maximum percentage of resource-consuming tasks.

J. Discussion

We discuss the future work of the proposed method. We have demonstrated the effectiveness of the proposed method. However, there are still challenges that we should explore for applying our method in a commercial environment or a future network.

We discuss the challenge of the applicability of the proposed method. We assume that the routes in the access network take the shortest path. Current wired access networks with optical fiber automatically select the shortest path from a few routing options. Current wireless access networks for cellular automatically select the best BS for each ED on the basis of the received signal power. Therefore, the proposed method is applicable in real-world scenarios of the current network. However, there are some challenges in applying the proposed method in a commercial environment or a future network.

The first challenge is scalability for edges. The proposed method may only be applicable to networks with EC servers with dozens of edges. We target commercial networks that cover a large area, such as networks in multiple regions or parts of a country. Such an access network includes more than 10 000 edges. Similar to other methods [1], [2], [3], [4], [5], [6], [7], [8] that can only handle a few dozen BSs at most, as described in Section II, our method also cannot control more than 10 000 edges. We deploy each agent at each edge (i.e., at each BS), and each agent learns cooperative control. Therefore, the agents should learn cooperative control among many agents in the mentioned environment. However, this is difficult to achieve with current technology and is one of the future works. As a possible solution, the combined method of Mean-field Game (MFG) theory and DRL may handle a large-scale task-offloading system with many BSs. MFG theory [35] studies strategic decision-making by small interacting agents in large populations, inspired by mean-field theory in physics. Each agent minimizes or maximizes the problem objective, taking into account the decisions of the other agents. Several studies have addressed network control methods that combine MFG theory and DRL, such as unmanned aerial vehicle (UAV) control [36], [37] and computation offloading in EC [38], [39], [40].

The next challenge is scalability for tasks. The processing time of the proposed method for determining the offload server

TABLE XVI
GENERALIZATION-PERFORMANCE EVALUATION FOR VARIOUS TASK TYPES

Case	Ave. Reward	Ave. Max. Node Util.	Ave. Max. Link Util.	Ave. Task-latency Eff.	Constraint Violation
Default	1.13 ± 0.21	0.52 ± 0.06	0.69 ± 0.02	0.35 ± 0.00	2.1 ± 2.5
Task 2–30%	1.03 ± 0.26	0.54 ± 0.05	0.69 ± 0.03	0.36 ± 0.00	2.8 ± 2.9
Task 2–40%	1.03 ± 0.24	0.55 ± 0.08	0.70 ± 0.03	0.36 ± 0.00	2.6 ± 2.7
Task 2–50%	0.94 ± 0.37	0.56 ± 0.05	0.70 ± 0.04	0.36 ± 0.00	4.1 ± 5.0
Task 3–22%	0.70 ± 0.36	0.58 ± 0.03	0.75 ± 0.03	0.37 ± 0.00	6.1 ± 4.8
Task 3–23%	0.34 ± 0.60	0.59 ± 0.03	0.79 ± 0.07	0.37 ± 0.00	10.6 ± 7.7
Task 3–25%	-0.36 ± 0.69	0.61 ± 0.04	0.88 ± 0.07	0.38 ± 0.00	19.6 ± 9.0
Task 4–30%	1.08 ± 0.23	0.49 ± 0.05	0.70 ± 0.03	0.38 ± 0.00	2.9 ± 3.1
Task 4–40%	0.88 ± 0.28	0.48 ± 0.07	0.73 ± 0.04	0.42 ± 0.00	5.3 ± 3.8
Task 4–50%	0.60 ± 0.38	0.43 ± 0.04	0.76 ± 0.03	0.44 ± 0.00	9.9 ± 5.1
All–15%	1.57 ± 0.11	0.35 ± 0.05	0.61 ± 0.03	0.31 ± 0.01	0.3 ± 1.1
All–25%	-0.14 ± 0.53	0.62 ± 0.04	0.85 ± 0.04	0.39 ± 0.00	16.6 ± 7.3
All–30%	-1.28 ± 0.47	0.63 ± 0.04	1.03 ± 0.08	0.42 ± 0.00	32.4 ± 6.3

is about 0.2 ms per task. Also, because the proposed method processes tasks sequentially, one at a time, it cannot handle thousands or millions of tasks per second required in real-world environments. Therefore, sufficient scalability for tasks is one of the future works. One possible solution is parallelization. Assigning many trained agents at each edge makes it possible to process many tasks simultaneously. Many trained agents working independently may cause new problems.

The other challenge is to optimize task offloading for an entire end-to-end network. Since task latency can be expressed as the sum of backbone and access networks, we can find near-optimal task offloading and routes between EDs and servers by optimizing the backbone and access networks independently. However, future networks will require more efficient optimal task offloading by considering the entire end-to-end multi-domain between EDs and servers, such as backbone network, access network, edge servers, and cloud servers. For this purpose, the proposed method should consider the latency and link quality of the wireless network between EDs and BSs. The difficulty in the wireless network is that the observed information is incomplete or uncertain. Several studies have addressed task offloading in the wireless network under imperfect channel state information (CSI) [41], [42], [43], [44], [45], [46]. In particular, security is one of the major concerns for mobile EC with incomplete CSI. Several studies have been conducted on security-aware task allocation under incomplete CSI [47], [48], [49]. In addition, several studies have been conducted on end-to-end network slicing or multi-domain network slicing to integrate the multiple controls [50], [51], [52]. Although much research has been done, optimizing task offloading in an end-to-end network remains an unsolved problem and one of the major challenges. By combining our method with above studies, the problem should be addressed in the future.

VII. CONCLUSION

We formulated an optimal task-offloading problem for multi-cloud and multi-edge networks considering network topology and bandwidth constraints. We proposed a cooperative task-offloading method that is based on cooperative multi-agent deep reinforcement learning (Coop-MADRL). This method can quickly achieve efficient task offloading by learning the relationship between network-demand patterns and optimal task offloading by using deep reinforcement learning in advance. This method also introduces a cooperative multi-agent technique, improving the efficiency of task offloading. Evaluations revealed that the proposed method can minimize network utilization and task latency while minimizing constraint violations in less than 1 ms in various network topologies. They also revealed that cooperative learning improves the efficiency of task offloading. We demonstrated that the proposed method can have generalization performance for various task types by pre-training with many resource-consuming tasks. We plan to evaluate the performance of the proposed method and conduct further detailed analysis in more complicated use cases or real-world applications. We also plan to improve the scalability and interpretability of the proposed method.

REFERENCES

[1] Y. Wang, X. Tao, X. Zhang, P. Zhang, and Y. T. Hou, "Cooperative task offloading in three-tier mobile computing networks: An ADMM framework," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2763–2776, Mar. 2019.

[2] H. Yuan and M. Zhou, "Profit-maximized collaborative computation offloading and resource allocation in distributed cloud and edge computing systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 18, no. 3, pp. 1277–1287, Jul. 2021.

[3] C. Kai, H. Zhou, Y. Yi, and W. Huang, "Collaborative cloud-edge-end task offloading in mobile-edge computing networks with limited communication capability," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 2, pp. 624–634, Jun. 2021.

[4] Y. Zhan, S. Guo, P. Li, and J. Zhang, "A deep reinforcement learning based offloading game in edge computing," *IEEE Trans. Comput.*, vol. 69, no. 6, pp. 883–893, Jun. 2020.

[5] D. C. Nguyen, P. N. Pathirana, M. Ding, and A. Seneviratne, "Deep reinforcement learning for collaborative offloading in heterogeneous edge networks," in *Proc. IEEE/ACM CCGrid*, 2021, pp. 297–303.

[6] W. Hou, H. Wen, H. Song, W. Lei, and W. Zhang, "Multiagent deep reinforcement learning for task offloading and resource allocation in cybertwin-based networks," *IEEE Internet Things J.*, vol. 8, no. 22, pp. 16256–16268, Nov. 2021.

[7] S. Ding and D. Lin, "Multi-agent reinforcement learning for cooperative task offloading in distributed edge cloud computing," *IEICE Trans. Inf. Syst.*, vol. 105, no. 5, pp. 936–945, 2022.

[8] Y. Zhang, B. Di, Z. Zheng, J. Lin, and L. Song, "Distributed multi-cloud multi-access edge computing by multi-agent reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 20, no. 4, pp. 2565–2578, Apr. 2021.

[9] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 1998.

[10] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI*, 2016, pp. 2094–2100.

[11] A. Suzuki, R. Kawahara, M. Kobayashi, S. Harada, Y. Takahashi, and K. Ishibashi, "Extendable NFV-integrated control method using reinforcement learning," *IEICE Trans. Commun.*, vol. E103.B, no. 8, pp. 826–841, 2020.

[12] G. J. Laurent, L. Matignon, and N. L. Fort-Piat, "The world of independent learners is not Markovian," *Int. J. Knowl.-Based Intell. Eng. Syst.*, vol. 15, no. 1, pp. 55–64, 2011.

[13] A. Suzuki and M. Kobayashi, "Multi-agent deep reinforcement learning for cooperative offloading in cloud-edge computing," in *Proc. ICC*, 2022, pp. 1–7.

[14] E. Oki, *Linear Programming and Algorithms for Communication Networks*. Boca Raton, FL, USA: CRC Press, 2012.

[15] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," in *Handbook of Reinforcement Learning Control*. Cham, Switzerland: Springer, 2021, pp. 321–384.

[16] A. Oroojlooy and D. Hajinezhad, "A review of cooperative multi-agent deep reinforcement learning," *Appl. Intell.*, to be published.

[17] S. Gronauer and K. Diepold, "Multi-agent deep reinforcement learning: A survey," *Artif. Intell. Rev.*, vol. 55, pp. 895–943, Feb. 2022.

[18] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[19] M. J. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *Proc. AAAI Fall Symp. Ser.*, 2015, pp. 29–37.

[20] D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein, "The complexity of decentralized control of Markov decision processes," *Math. Oper. Res.*, vol. 27, no. 4, pp. 819–840, 2002.

[21] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*. Cham, Switzerland: Springer, 2016.

[22] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proc. ICML*, 1993, pp. 487–494.

[23] A. Tampuu et al., "Multiagent cooperation and competition with deep reinforcement learning," *PLoS ONE*, vol. 12, no. 4, 2017, Art. no. e0172395.

[24] P. Sunehag et al., "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proc. AAMAS*, 2018, pp. 2085–2087.

[25] A. Suzuki, R. Kawahara, and S. Harada, "Cooperative multi-agent deep reinforcement learning for dynamic virtual network allocation," in *Proc. ICCCN*, 2021, pp. 1–11.

[26] A. Suzuki, R. Kawahara, and S. Harada, "Cooperative multi-agent deep reinforcement learning for dynamic virtual network allocation with traffic fluctuations," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 1982–2000, Sep. 2022.

[27] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. NIPS Workshop*, 2014, pp. 1–9.

[28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

[29] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NIPS*, 2019, pp. 8026–8037.

[30] M. Samvelyan et al., "The StarCraft multi-agent challenge," 2019, *arXiv:1902.04043*.

[31] J. Hu, S. Jiang, S. A. Harding, H. Wu, and S.-W. Liao, "Rethinking the implementation tricks and monotonicity constraint in cooperative multi-agent reinforcement learning," 2021, *arXiv:2102.03479*.

[32] "GLPK." 2022. [Online]. Available: https://www.gnu.org/software/glpk

[33] R. Summerhill, "The new Internet2 network," in *Proc. GLIF Meeting*, 2006, pp. 1–26.

[34] S. Orlowski, R. Wessäly, M. Pióro, and A. Tomaszewski, "SNDlib 1.0—Survivable network design library," *Netw. Int. J.*, vol. 55, no. 3, pp. 276–286, 2010.

[35] A. Vasiliadis, "An introduction to mean field games using probabilistic methods," 2019, *arXiv:1907.01411*.

[36] D. Chen, Q. Qi, Z. Zhuang, J. Wang, J. Liao, and Z. Han, "Mean field deep reinforcement learning for fair and efficient UAV control," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 813–828, Jan. 2021.

[37] L. Li, Q. Cheng, K. Xue, C. Yang, and Z. Han, "Downlink transmit power control in ultra-dense UAV network based on mean field game and deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15594–15605, Dec. 2020.

[38] D. Shi, H. Gao, L. Wang, M. Pan, Z. Han, and H. V. Poor, "Mean field game guided deep reinforcement learning for task placement in cooperative multiaccess edge computing," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9330–9340, Oct. 2020.

[39] R. A. Banez et al., "Mean-field-type game-based computation offloading in multi-access edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 8366–8381, Dec. 2020.

[40] R. Zheng, H. Wang, M. De Mari, M. Cui, X. Chu, and T. Q. S. Quek, "Dynamic computation offloading in ultra-dense networks based on mean field games," *IEEE Trans. Wireless Commun.*, vol. 20, no. 10, pp. 6551–6565, Oct. 2021.

[41] Z. Jian, W. Muqing, and Z. Min, "Joint computation offloading and resource allocation in C-RAN with MEC based on spectrum efficiency," *IEEE Access*, vol. 7, pp. 79056–79068, 2019.

[42] S. Li, Y. Tao, X. Qin, L. Liu, Z. Zhang, and P. Zhang, "Energy-aware mobile edge computation offloading for IoT over heterogenous networks," *IEEE Access*, vol. 7, pp. 13092–13105, 2019.

[43] L. Tang and H. Hu, "Computation offloading and resource allocation for the Internet of Things in energy-constrained MEC-enabled HetNets," *IEEE Access*, vol. 8, pp. 47509–47521, 2020.

[44] B. Liu, J. Song, J. Wang, H. Sun, and Q. Wang, "Robust secure wireless powered MISO cognitive mobile edge computing," *IEEE Access*, vol. 8, pp. 62356–62366, 2020.

[45] L. Wang, H. Shao, J. Li, X. Wen, and Z. Lu, "Optimal multi-user computation offloading strategy for wireless powered sensor networks," *IEEE Access*, vol. 8, pp. 35150–35160, 2020.

[46] K. Wang, Y. Zhou, Q. Wu, W. Chen, and Y. Yang, "Task offloading in hybrid intelligent reflecting surface and massive MIMO relay networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 6, pp. 3648–3663, Jun. 2022.

[47] J. Xu and J. Yao, "Exploiting physical-layer security for multiuser multicarrier computation offloading," *IEEE Wireless Commun. Lett.*, vol. 8, no. 1, pp. 9–12, Feb. 2019.

[48] X. Lai, L. Fan, X. Lei, Y. Deng, G. K. Karagiannidis, and A. Nallanathan, "Secure mobile edge computing networks in the presence of multiple eavesdroppers," *IEEE Trans. Commun.*, vol. 70, no. 1, pp. 500–513, Jan. 2022.

[49] Y. Guo, R. Zhao, S. Lai, L. Fan, X. Lei, and G. K. Karagiannidis, "Distributed machine learning for multiuser mobile edge computing systems," *IEEE J. Sel. Topics Signal Process.*, vol. 16, no. 3, pp. 460–473, Apr. 2022.

[50] I. Kovacevic, A. S. Shafigh, S. Glisic, B. Lorenzo, and E. Hossain, "Multi-domain network slicing with latency equalization," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2182–2196, Dec. 2020.

[51] H. Bai, Y. Zhang, Z. Zhang, and S. Yuan, "Latency equalization policy of end-to-end network slicing based on reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 88–103, Mar. 2023.

[52] Y. Chiang, C.-H. Hsu, G.-H. Chen, and H.-Y. Wei, "Deep Q-learning-based dynamic network slicing and task offloading in edge network," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 369–384, Mar. 2023.

Akito Suzuki received the B.E. degree in electronic and physical systems and the M.E. degree in nanoscience and engineering from Waseda University, Tokyo, Japan, in 2013 and 2015, respectively. He is currently pursuing the Ph.D. degree in informatics with Kyoto University, Japan. He is an Associate Distinguished Researcher with NTT Network Service Systems Laboratories. In 2015, he joined NTT Laboratories, he has researched network control and traffic engineering. His research interests include resource allocation, mathematical optimization, and reinforcement learning.

Masahiro Kobayashi received the B.S. and M.S. degrees in information science from Tohoku University, Miyagi, in 2007 and 2009, respectively. He joined NTT in 2009, where he is currently studying optimal resource control in the virtualized communication network with NTT Network Service Systems Laboratories. He is a Research Engineer with NTT Network Service Systems Laboratories. He is a member of the Institute of Electronics, Information and Communication Engineers.

Eiji Oki (Fellow, IEEE) received the B.E. and M.E. degrees in instrumentation engineering and the Ph.D. degree in electrical engineering from Keio University, Yokohama, Japan, in 1991, 1993, and 1999, respectively. He was with Nippon Telegraph and Telephone Corporation Laboratories, Tokyo, from 1993 to 2008, and The University of Electro-Communications, Tokyo, from 2008 to 2017. From 2000 to 2001, he was a Visiting Scholar with Polytechnic University, Brooklyn, NY, USA. In 2017, he joined Kyoto University, Japan, where he is currently a Professor. His research interests include routing, switching, protocols, optimization, and traffic engineering in communication and information networks.